UNIVERSITY OF PLYMOUTH

# PUSL3190 Computing Project

# Final Report

An Automated Vehicle Attendance and Movement Analytics System for Industrial Facilities Using Computer Vision

**Supervisor:** Mr. Madusanka Mithrananda

**Name:** Garuka Assalaarachchi

**Plymouth Index Number:** 10952592

**Degree Program:** BSc. in Software Engineering

# Declaration of Originality

I hereby declare that this thesis, submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Software Engineering at the University of Plymouth (Sri Lanka Campus), is my own original work.

(i) This work has not previously been submitted for any other degree or qualification at this or any other institution.

(ii) All sources consulted have been acknowledged by full citation in accordance with the Harvard referencing convention.

(iii) All software code written as part of this project is my own work, except where explicitly attributed to third-party libraries identified in requirements.txt and the relevant sections of this report.

Signature: _____

Date: April 2026

# Acknowledgements

I wish to express my sincere gratitude to my project supervisor for their guidance, critical feedback, and patience throughout this project. Their direction in refocusing the scope of this work toward the vehicle attendance use case was instrumental in producing a more coherent and practically relevant contribution.

I am grateful to the academic staff of the Department of Computing at the University of Plymouth (Sri Lanka Campus) for their support during the 2025/2026 cohort, and to my peers for their encouragement during the development and evaluation phases.

I thank the staff at Colombo Dockyard PLC for providing the operational context that motivated this research and for sharing operational data that grounded the problem analysis in Section 3.

# Abstract

Industrial facilities that rely on contractor vehicle fleets face a persistent challenge in accurately recording vehicle attendance. Manual logbooks are error-prone and falsifiable, while radio-frequency identification gate systems experience read failures of 15 to 20 percent under Sri Lankan port field conditions, producing records that are incomplete and indefensible in payroll disputes (Delen et al., 2011). Time-theft and buddy-punching fraud in manual attendance systems cost organisations an estimated seven percent of annual payroll (American Payroll Association, 2023), while unmonitored fleet fuel usage results in further unaccounted operational expenditure (Raza et al., 2022).

This project designed, implemented, and evaluated an automated vehicle attendance and analytics system that uses computer vision to identify vehicles by their licence plates at facility gates, with no hardware required on individual vehicles. A custom-trained two-stage image recognition pipeline was developed for Sri Lankan alphanumeric plates, incorporating a domain-specific weighted error-correction algorithm (LPM-MLED) to resolve visually ambiguous characters, following the framework of Kechagias-Stamatis et al. (2022) and Islam et al. (2020). CLAHE contrast enhancement addresses the significant illumination variation of Sri Lankan industrial environments (Suleman et al., 2022; Al-Dabbagh et al., 2024).

A shift-aware attendance engine records arrival and departure events, computes dwell time, flags late arrivals and early departures, and handles unregistered vehicles through a structured operator alert workflow. An enterprise analytics layer delivers four research-grounded management report types: a payroll accuracy report (Rahman et al., 2023), an OHS compliance report (Pawar et al., 2021), a fuel accountability report (Raza et al., 2022), and a post-incident gate rejection audit log (Yue et al., 2016). All records are protected by a cryptographic SHA-256 hash chain with photographic plate-crop evidence and a tamper-evident admin audit log.

Evaluation against 150 physical test plates and 150 scripted attendance scenarios achieved 91.3% end-to-end plate recognition accuracy — a 20.3 percentage-point improvement over EasyOCR — and 97.3% attendance event accuracy, a 12 to 18 percentage-point improvement over the existing RFID gate system. Gate event processing completed in under 300 milliseconds at the 95th percentile. The per-gate hardware cost of approximately 85,000 Sri Lankan Rupees is 40 times lower than commercially available alternatives, demonstrating that accurate, auditable, and analytically actionable vehicle attendance management is achievable at small enterprise scale.

# Table of Contents

# List of Abbreviations

| Abbreviation | Definition |
|---|---|
| ALPR | Automatic Licence Plate Recognition |
| ANPR | Automatic Number Plate Recognition |
| API | Application Programming Interface |
| APA | American Payroll Association |
| CLAHE | Contrast Limited Adaptive Histogram Equalisation |
| CNN | Convolutional Neural Network |
| CSV | Comma-Separated Values |
| CTC | Connectionist Temporal Classification |
| EPC | Electronic Product Code |
| FP16 | 16-bit Floating Point (half-precision) |
| FR | Functional Requirement |
| FYP | Final Year Project |
| HR | Human Resources |
| ISO | International Organisation for Standardisation |
| LKR | Sri Lankan Rupee |
| LPM-MLED | Licence Plate Matching by Minimum Levenshtein Edit Distance |
| mAP | mean Average Precision |
| NFR | Non-Functional Requirement |
| NTP | Network Time Protocol |
| OBD-II | On-Board Diagnostics II |
| OCR | Optical Character Recognition |
| OHS | Occupational Health and Safety |
| p95 | 95th Percentile |
| PDPA | Personal Data Protection Act |
| PDF | Portable Document Format |
| RBAC | Role-Based Access Control |
| RFID | Radio Frequency Identification |
| SHA-256 | Secure Hash Algorithm 256-bit |
| SME | Small and Medium Enterprise |

| Abbreviation | Definition |
|---|---|
| SQLite | Self-contained SQL Database Engine |
| SSE | Server-Sent Events |
| SUS | System Usability Scale |
| T&A | Time and Attendance |
| UHF | Ultra-High Frequency |
| VAAS | Vehicle Attendance and Analytics System |
| WAL | Write-Ahead Log |
| YOLO | You Only Look Once |

# 1 Introduction

## 1.1 Background

The management of vehicle access at large industrial facilities — ports, shipyards, manufacturing complexes, and logistics hubs — is a fundamental operational control problem. Unlike pedestrian workers who present a biometric credential at a turnstile, vehicle-borne workers and contractors enter as an inseparable human-vehicle unit: the vehicle is the attendance record, the billing unit, and the primary safety identifier (Nambiar, 2021). Systems that are adequate for pedestrian workforce management fail systematically in this context.

Colombo Dockyard PLC (CDL; Colombo Stock Exchange ticker: DOCK.N0000) is Sri Lanka's largest and oldest ship repair and shipbuilding enterprise, incorporated in 1974 and majority-owned by Onomichi Dockyard Co. Ltd of Japan (approximately 51 percent equity stake) since the joint venture was formalised in 1992 (CDL Annual Report, 2024) (CDL Annual Report, 2024). CDL operates continuously across three eight-hour shifts within the Port of Colombo, Sri Lanka's principal commercial seaport, under certification to ISO 9001 (quality management), ISO 14001 (environmental management), and ISO 45001 (occupational health and safety). Its four graving drydocks accommodate vessels up to 125,000 DWT; the facility processes over 200 ships annually for international shipping companies, the Sri Lanka Navy, and offshore energy clients. CDL operates through two specialist subsidiaries — Dockyard General Engineering Services (DGES) for heavy civil and mechanical engineering, and Dockyard Total Solutions (DTS) for integrated facilities management — each contributing further vehicular and personnel access requirements to the gate system. In 2024 alone, CDL's in-house Training Centre inducted 2,507 new employees and subcontractor personnel through safety certification programmes, underscoring the operational scale of the facility's HSE compliance burden (CDL Annual Report, 2024). Classification society surveyors from Lloyd's Register, Bureau Veritas, ClassNK, and DNV are regular gate visitors, requiring a visitor exception workflow alongside routine contractor gate management (CDL Annual Report, 2024). Its workforce includes approximately 1,400 to 3,000 permanent employees and trainees together with a highly fluid population of subcontractor personnel whose numbers peak during major vessel refit cycles. CDL identified a specific business requirement: to create an objective, automated record of which employee personal vehicles attend the facility, by shift, so that a contractual personal-vehicle travel allowance could be calculated fairly, verifiably, and without the falsification risk of paper logbooks (operational observation during internship, CDL, 2024).

This requirement — simple in statement — exposes a fundamental technology gap. The dominant alternatives each fail CDL's operational constraints:

- Manual paper logbooks: 3 to 7 percent data-entry error rate (Ko, 2011); easily falsified; provide no structured output for payroll calculations.
- Biometric terminals: designed for pedestrian workers at fixed access points; architecturally inapplicable to vehicle-borne attendance (Bhatt et al., 2019; Nambiar, 2021).

- RFID gate readers: read success rates fall to 79 to 85 percent in humid, coastal industrial environments analogous to CDL's Port of Colombo site, producing systematically incomplete records (Delen et al., 2011; Thiesse et al., 2007); passive UHF tags are additionally vulnerable to cloning (Juels, 2006).
- Commercial ALPR systems: engineered for NATO-format plates; priced at USD 10,000 to 25,000 per gate (Genetec, 2023); provide logging but not attendance lifecycle management, shift tracking, or management analytics.

Automatic Licence Plate Recognition (ALPR) using deep learning resolves the core identification problem by treating the licence plate as a persistent, hardware-free vehicle identifier. The third generation of ALPR technology — exemplified by YOLOv8 single-stage detection (Jocher et al., 2023) — delivers real-time inference on edge hardware, achieving greater than 93 percent recognition accuracy across adverse conditions (Al-Dabbagh et al., 2024; Sabir et al., 2023). However, ALPR alone is not an attendance system. Transforming plate recognition into an operationally useful vehicle attendance and analytics platform — one that handles Sri Lankan alphanumeric plate conventions, enforces shift-based compliance rules, produces auditable payroll data, and attributes attendance to specific drydock projects — requires substantial additional engineering (Kechagias-Stamatis et al., 2022; Rahman et al., 2023).

This project addresses that engineering problem directly, with CDL as the primary deployment context. The resulting system, called the Vehicle Attendance and Analytics System (VAAS), is designed to serve CDL's incentive-programme requirement while providing the full operational analytical capability — payroll verification, OHS compliance, fuel accountability, subcontractor billing audit — that CDL's operational complexity demands (Pawar et al., 2021; Raza et al., 2022).

## 1.2 Problem Statement

CDL and comparable Sri Lankan industrial facilities face a compound four-layer vehicle attendance problem:

Architectural mismatch: conventional T&A systems cannot record vehicle-level attendance; the vehicle — not the person — is the operational unit at CDL's gate (Nambiar, 2021; Bhatt et al., 2019).

Data integrity failure: RFID readers deliver 15 to 20 percent non-read rates under Sri Lankan port conditions; manual fallbacks introduce 3 to 7 percent transcription errors (Delen et al., 2011; Ko, 2011); neither system supports post-incident reconstruction.

Analytics gap: no existing deployed system links vehicle attendance to a specific drydock project, preventing project-based incentive calculation and subcontractor billing verification.

Cost barrier: commercial ALPR systems at USD 10,000 to 25,000 per gate (Genetec, 2023) are cost-prohibitive for the CDL deployment model; GPS telematics at USD 33 per vehicle per month (Samsara, 2023) require per-vehicle hardware incompatible with CDL's subcontractor fleet structure.

## 1.3 Aim and Objectives

**Aim:** To design, implement, and evaluate a computer-vision-based vehicle attendance and movement analytics system, deployed in the operational context of Colombo Dockyard PLC, that provides accurate, shift-aware, project-attributed, auditable, and analytically actionable records of vehicle presence at industrial facility gates, at a cost accessible to Sri Lankan industrial enterprises.

The following objectives were set to operationalise this aim:

- **O1 — High-Accuracy Plate Recognition:** Develop a robust ALPR pipeline using YOLOv8 plate detection, CLAHE contrast enhancement (Suleman et al., 2022; Al-Dabbagh et al., 2024), and a custom 37-class character classifier incorporating the LPM-MLED post-correction algorithm (Kechagias-Stamatis et al., 2022; Islam et al., 2020) targeting ≥ 90% end-to-end accuracy on Sri Lankan alphanumeric plates.
- **O2 — CDL-Aware Attendance Engine:** Implement a shift-aware attendance engine covering CDL's three-shift pattern (07:00–15:00, 15:00–23:00, 23:00–07:00) with clock-in/clock-out event capture, dwell-time computation, late-arrival flagging, project-level attendance attribution, driver-vehicle assignment tracking, and a structured exception-handling workflow for unregistered and visitor vehicles.
- **O3 — Enterprise Analytics and CDL Specialisation:** Build an analytics dashboard delivering four research-grounded management report types — payroll accuracy (Rahman et al., 2023), OHS compliance (Pawar et al., 2021), fuel accountability (Raza et al., 2022), and gate rejection audit (Yue et al., 2016) — extended with CDL-specific project attendance summaries and subcontractor billing verification.
- **O4 — Tamper-Evident Audit Trail:** Implement a cryptographically protected audit trail using chained SHA-256 row hashing and base64-encoded plate-crop image evidence (Yue et al., 2016; Azaria et al., 2016), with a separate admin audit log recording all CRUD operations.

## 1.4 Scope and Delimitations

This project implements and evaluates VAAS within a controlled simulation of the CDL gate environment using a 1:18 scale tabletop testbed with two functional gate nodes. The scope includes:

- ALPR pipeline for Sri Lankan alphanumeric plates, covering all provincial and commercial plate formats issued under current SL registration conventions.
- Attendance lifecycle management for CDL's three-shift pattern, including grace-period enforcement (15 minutes to accommodate Port of Colombo entry queues) and exception workflows.
- Project-level attribution (vessel name, drydock zone) and subcontractor company management (src/projects.py, §6.5).
- Four enterprise analytics reports with CSV and PDF export.
- SHA-256 tamper-evident audit chain with row-reordering attack prevention.

The following are explicitly out of scope for this submission:

- Live integration with CDL's HR, ERP, or payroll systems.
- Mobile client applications.
- Real-world physical deployment at CDL (deferred to future work, Section 11).
- ALPR for non-Sri Lankan plate formats.

## 1.5 Structure of This Report

Section 2 reviews literature across time-and-attendance management, RFID vehicle tracking, ALPR, weighted edit-distance post-correction, fleet analytics, and audit integrity, prioritising publications from 2020 onwards. Section 3 presents a structured analysis of CDL's operational context: stakeholders, existing and proposed processes, feasibility, and cost-benefit. Section 4 defines the research methodology and system requirements. Section 5 presents the system architecture. Section 6 details the implementation. Section 7 reports testing and evaluation. Section 8 discusses findings. Section 9 is the end-project report. Section 10 is the post-mortem. Section 11 concludes. Appendices provide the complete test output, database schema, user guide, and source code repository link.

# 2 Literature Review

## 2.1 Introduction

This chapter reviews literature across five intersecting domains relevant to the design of VAAS: time-and-attendance management technologies; RFID vehicle identification and its documented limitations; the evolution of ALPR to YOLOv8-based pipelines; weighted edit-distance post-correction for OCR output; and fleet management analytics. The chapter concludes with a structured gap analysis. Publications from 2020 or later are preferred; earlier foundational works are cited only where no recent equivalent exists.

## 2.2 Time-and-Attendance Management

Time-and-attendance (T&A) management is well-studied across three technology generations:

- **Biometric terminals (fingerprint, iris, face):** Provide strong individual identity assurance (Bhatt et al., 2019) but assume the attendance unit is a person at a fixed terminal — an assumption violated in vehicle-centric environments where the vehicle is the operational and financial unit (Nambiar, 2021).
- **Proximity-card / RFID systems:** Extend biometric-style logic to vehicles via UHF tags, but suffer the field reliability failures detailed in Section 2.3.
- **GPS telematics platforms:** Provide rich analytics but require per-vehicle OBD-II or satellite hardware at USD 33 or more per vehicle per month (Samsara, 2023) — prohibitive for CDL's subcontractor fleet model, where vehicles are not CDL assets (Nambiar, 2021).

Manual fallbacks — paper logbooks and supervisor sign-off sheets — introduce well-documented accuracy problems. Ko (2011) estimated a 3 to 7 percent data-entry error rate; the American Payroll Association (2023) reports that timesheet falsification and buddy-punching affect the majority of businesses and inflate payroll costs by up to seven percent of total payroll. Rahman et al. (2023) demonstrated that sensor-derived records reduce payroll dispute resolution time by 68 percent by removing reliance on self-reported timesheets.

## 2.3 RFID Vehicle Identification: Field Performance

Ultra-high frequency RFID has been the dominant technology for automated vehicle gate management since the early 2000s. Field performance in environments analogous to CDL's presents persistent reliability challenges:

- Delen et al. (2011) surveyed RFID deployments at three US seaport facilities and documented mean gate-read success rates of 84.3 percent under peak throughput, falling to 79.1 percent during rain — consistent with the target facility's documented non-read rate of 15 to 20 percent.
- Thiesse et al. (2007) found that tag-to-reader coupling degraded by up to 23 percent in humid, salt-laden coastal atmospheres directly comparable to Colombo Harbour, confirming RFID's unsuitability for CDL's environment.

- Juels (2006) identified that passive UHF tags can be cloned — a fraudulent vehicle can carry a transponder programmed with a registered vehicle's EPC code, a security vulnerability directly addressed by VAAS's plate-crop photographic evidence and SHA-256 chain (FR-05).

## 2.4 Automatic Licence Plate Recognition

### 2.4.1 Three Generations of ALPR

ALPR research spans three technological generations:

- **First generation (1990s–2005):** Classical edge detection and SVM classification; 70 to 80 percent accuracy under controlled conditions; brittle under illumination variation and motion blur (Jiao et al., 2009).
- **Second generation (2005–2017):** CNN-learned representations; Shi et al. (2016) proposed CTC for end-to-end sequence recognition; Li et al. (2019) achieved 95.2 percent on the CCPD Chinese plate dataset.
- **Third generation (2017–present):** Single-stage detectors; YOLOv8 (Jocher et al., 2023) reports 37.3 mAP on COCO at 18.4 ms CPU inference. Al-Dabbagh et al. (2024) achieved greater than 93 percent recognition across night-time and rainy scenarios; Sabir et al. (2023) achieved mAP@0.5 of 94.7 percent on plate detection.

For Sri Lankan plates specifically, Kalansuriya et al. (2014) achieved only 78 percent accuracy using classical methods, and no subsequent deep-learning study targeting Sri Lankan plates was identified prior to this project — confirming the need for the custom pipeline developed in VAAS.

### 2.4.2 CLAHE Contrast Enhancement

Contrast Limited Adaptive Histogram Equalisation (CLAHE) addresses illumination variation systematically:

- Suleman et al. (2022) demonstrated a +8.4 percentage-point accuracy improvement when CLAHE was applied before character classification in underexposed and overexposed plate crops.
- Dewi et al. (2022) validated CLAHE for plates under fog and overcast conditions (90 percent accuracy vs. 74 percent unprocessed).
- Al-Dabbagh et al. (2024) incorporated CLAHE into their night-time and rainy ALPR pipeline, achieving greater than 93 percent overall recognition.

These results motivate the CLAHE configuration adopted in VAAS (clip limit 3.0, tile size 8×8, applied only to the L channel of the LAB colour space), as detailed in Section 6.2.

## 2.5 Post-Correction of OCR Output: Weighted Edit Distance

Neural ALPR pipelines systematically confuse visually similar character pairs (0/O, 1/I, 5/S, 8/B) on Sri Lankan plates. Standard Levenshtein edit distance (Levenshtein, 1966) applies uniform cost, which is suboptimal where $0 \rightarrow O$ substitution is far more probable than $0 \rightarrow K$. Weighted edit distance (Brill and Moore, 2000) assigns domain-specific substitution costs:

- Islam et al. (2020) applied weighted edit distance to Bangladeshi plate post-correction, reporting a 9.3 percentage-point accuracy improvement over an unweighted baseline.
- Wang et al. (2022) extended the approach to Chinese plates using a Bayesian confusion matrix.
- Kechagias-Stamatis et al. (2022) validated weighted edit distance for European country-code recognition, demonstrating that character-similarity weights substantially outperform both standard edit distance and character-frequency priors.

The LPM-MLED algorithm in VAAS follows this lineage with confusion-pair costs (0.1 for 8/B, 0/O, 1/I, 5/S; 1.0 for all other pairs) normalised by max plate length, as described in Section 6.2.

## 2.6 Fleet Management Analytics

The literature identifies four management report types that transform raw attendance records into actionable operational intelligence:

- **Payroll accuracy:** Rahman et al. (2023) demonstrated that sensor-derived attendance records reduce payroll dispute resolution time by 68 percent and eliminate the timesheet falsification vulnerability estimated to affect 75 percent of businesses (APA, 2023).
- **OHS compliance:** Pawar et al. (2021) found that proactive digital OHS monitoring — identifying unassigned vehicles and high-overstay patterns — reduced incident rates by 34 percent in a comparable multi-vehicle industrial deployment.
- **Fuel accountability:** Raza et al. (2022) demonstrated that dwell-time-based fuel consumption proxies rank vehicles by consumption tier with 87 percent accuracy against metered readings, recovering 12 to 18 percent of previously unaccounted fuel expenditure.
- **Post-incident audit:** Yue et al. (2016) established the foundational architectural requirement for SHA-256 hash chaining to enable tamper-evident post-incident record reconstruction.

## 2.7 Audit Integrity and Tamper-Evident Records

Sri Lanka's Payment of Gratuity Act No. 12 of 1983 and Employees' Provident Fund Act require verifiable attendance records for all worker categories, including contractor personnel. Conventional relational databases provide no inherent protection against retrospective record modification. Hash-chaining, as proposed by Yue et al. (2016), ensures that any alteration to a historical record invalidates the hash of every subsequent record, making tampering algorithmically detectable without distributed consensus infrastructure. Azaria et al. (2016) demonstrated that chain integrity can be verified by any party holding only the genesis hash. Microsoft (2022) independently validated this pattern in SQL Server 2022 Ledger — confirming its enterprise-grade suitability.

VAAS extends the standard hash-chain with the inclusion of the row's primary-key ID in the hash payload (a countermeasure against row-reordering attacks, identified during security

review; see Section 6.6), and with base64-encoded plate-crop photographic evidence stored alongside each access log record.

## 2.8 Summary and Gap Analysis

Table 2.1 positions VAAS against five existing technology classes across the six dimensions that directly map to CDL's operational requirements.

| Technology | Vehicle T&A | SL Plates | CDL Shifts | Project Analytics | Audit Chain | C |
|---|---|---|---|---|---|---|
| Biometric T&A | No | N/A | No | No | Partial | L |
| RFID UHF | Partial (15–20% errors) | N/A | No | No | No | M |
| GPS Telematics | Yes | N/A | No | No | Partial | H v |
| Commercial ALPR | Partial | No | No | No | No | H 1 |
| Open-source ALPR | No | No | No | No | No | L |
| VAAS (this project) | Yes | Yes | Yes (3-shift) | Yes (projects.py) | Yes (SHA-256) | L ( L |

*Table 2.1 — Technology Positioning Across CDL Operational Requirements*

No existing system satisfies all six dimensions simultaneously within CDL's cost envelope. VAAS is designed to close this gap precisely, grounding each component in peer-reviewed operational research.

# 3 Analysis

## 3.1 Introduction

A rigorous analysis phase grounded the design of VAAS in CDL's specific operational context and stakeholder requirements (IEEE, 1998). This chapter presents: a CDL stakeholder analysis; the existing (AS-IS) and proposed (TO-BE) gate management processes; a four-dimension feasibility analysis; a structured cost-benefit analysis; and a requirements traceability matrix.

## 3.2 Stakeholder Analysis

The following primary stakeholders were identified through operational observation at CDL (2024) and review of fleet management literature (Pawar et al., 2021; Rahman et al., 2023):

| Stakeholder | Department / Role | Primary Need | Key Risk Mitigated |
|---|---|---|---|
| HR / Payroll Manager | Human Resources | Auditable vehicle-hour records for personal-vehicle allowance calculation (FR-06) | Timesheet falsification; unresolvable disputes (APA, 2023) |
| Security Manager | Site Security | Real-time gate log; gate rejection audit (FR-09) | Incomplete records; inability to reconstruct post-incident |
| Drydock Manager | Vessel Project Management | Project-level attendance and zone occupancy per drydock project (FR-11, FR-12); real-time headcount per graving dock | Contractor no-shows; unverifiable subcontractor headcount; zone capacity breaches |
| Subcontractor Liaison | Procurement / Contracts | Gate-log billing hours per approved sub-contracting firm; approved company register (FR-11) | Invoice discrepancies; unapproved vehicles on-site; suspended company vehicles admitted |
| Fleet / Operations Manager | Operations | Occupancy reports; fuel accountability (FR-08) | Fuel discrepancies; shift window violations |
| OHS Compliance Officer | Safety | OHS compliance report (FR-07); driver-vehicle assignment (FR-04) | Unassigned vehicles; unattributable incidents (Pawar et al., 2021) |
| Gate Operator | Security / Operations | Instant plate status; exception workflow on unregistered vehicles | System latency; false rejections |
| Port Facility Security Officer (PFSO) | Maritime Security | ISPS Code enforcement; MARSEC level control; anti-passback gate state; watchlist alert feed (FR-03, FR-05) | Unauthorised vessel-side access; credential cloning; ISPS audit failure |
| External Auditor | Regulatory / Finance | SHA-256 chain integrity; admin audit log (FR-10) | Retrospective tampering; record gaps |

*Table 3.1 — CDL Stakeholder Analysis Matrix*

## 3.3 Business Process Analysis

### 3.3.1 AS-IS Process: Existing CDL Gate Management

CDL's current gate management process was documented through operational observation during an internship period in 2024. Key process steps and identified failure points are:

Vehicle arrives at Security Checkpoint; officer manually checks the contractor register and records plate number, time, and company name in a paper logbook.

Where installed, RFID reader attempts a tag read. Read is not cross-referenced in real time; failures are assumed as a successful entry in the daily count (Delen et al., 2011).

No mechanism exists to link a vehicle visit to the specific drydock project it is supporting. Subcontractor billing is based on invoiced hours, with no gate-log basis for verification.

At shift boundary (07:00 / 15:00 / 23:00), vehicles present in one shift are manually carried over to the next in a paper handover register.

At day end, a supervisor manually compiles departure counts; dwell time is not computed.

Documented failure points: 3 to 7 percent transcription error rate (Ko, 2011); 15 to 20 percent RFID non-reads systematically over-reporting attendance (Delen et al., 2011); no structured exception workflow for unregistered or visitor vehicles; no project-level attribution; no subcontractor billing audit capability.

### 3.3.2 TO-BE Process: VAAS-Mediated CDL Gate Management

The proposed VAAS process replaces the manual record at every stage:

Vehicle approaches gate; VAAS camera captures video at ≥ 15 fps. YOLOv8 detects and crops the plate region in real time (Jocher et al., 2023).

CLAHE contrast enhancement (clip limit 3.0, tile 8×8) applied to the L channel of the LAB colour space (Suleman et al., 2022; Al-Dabbagh et al., 2024).

Custom 37-class character classifier produces a raw plate string; LPM-MLED resolves visually ambiguous characters (Kechagias-Stamatis et al., 2022; Islam et al., 2020).

Attendance engine evaluates: registration status (ACTIVE / SUSPENDED / EXPIRED), shift eligibility (CDL three-shift pattern, 15-minute grace period), project assignment (if applicable). If ACTIVE and eligible: clock-in event written to access_log with SHA-256 hash, base64 plate-crop image, zone_id, and project_code; barrier opens.

On exit: as above with direction EXIT; dwell_time_seconds computed automatically.

CDL Specialisation Layer: drydock manager accesses project attendance summary; subcontractor liaison accesses billing hours verification; HR queries personal-vehicle attendance days for incentive calculation — all via secured web dashboard.

All administrative CRUD operations logged to admin_audit_log (FR-10).

## 3.4 Feasibility Analysis

### 3.4.1 Technical Feasibility

Core components rely on mature, peer-reviewed algorithms and open-source frameworks. YOLOv8 is production-tested across thousands of industrial deployments (Jocher et al., 2023). On the development hardware (NVIDIA RTX 3050, FP16 mode), inference runs at 45 fps — a 3× safety margin above the 15 fps minimum required for gate-speed plate capture. SQLite WAL mode supports concurrent read access from the web dashboard without blocking gate event writes, satisfying the concurrency requirements of CDL's multi-gate topology.

### 3.4.2 Economic Feasibility

Per-gate hardware is estimated at 85,000 LKR (approximately USD 265 at 2024 exchange rates): IP camera (35,000 LKR), edge computing device (40,000 LKR), Arduino Nano barrier controller (10,000 LKR). This is 40× lower than commercial ALPR systems at USD 10,000 to 25,000 per gate (Genetec, 2023) and avoids the per-vehicle subscription model of GPS telematics (Samsara, 2023). Payback is estimated at under three months against CDL's documented fuel discrepancy of 2.7 to 3.6 million LKR annually (operational observation, CDL, 2024).

### 3.4.3 Legal and Ethical Feasibility

Licence plates are publicly visible identifiers. Under the Sri Lanka Personal Data Protection Act No. 9 of 2022 (PDPA 2022, Section 6(2)(b)), processing of publicly visible identifiers by an entity with a legitimate operational interest does not require individual consent. Plate-crop images are subject to a configurable 90-day retention policy with automatic purging (NFR-03). No biometric data is collected or stored. VAAS's audit trail satisfies evidentiary requirements under the Payment of Gratuity Act No. 12 of 1983.

### 3.4.4 Operational Feasibility

CDL gate operators currently use paper logbooks; the VAAS web dashboard requires only browser access, eliminating the need for specialist software training. The exception-handling workflow (unregistered plate → operator alert → approve / reject decision with one click) mirrors existing manual exception protocols. System-level training is estimated at two hours per gate operator, aligned with CDL's standard induction programme for new gate systems.

## 3.5 Cost-Benefit Analysis

| Item | Amount (LKR) | Notes |
|---|---|---|
| Per-gate hardware (camera, compute, Arduino) | 85,000 | One-time capital — see §3.4.2 |
| Installation and configuration per gate | 20,000 | One day on-site labour |
| Annual maintenance per gate (estimate) | 15,000 | Preventive inspection, consumables |
| Three-year total cost (two gates) | 310,000 | Capital + maintenance |
| Annual fuel discrepancy reduction (50% capture est.) | 1,350,000–1,800,000 | Documented gap: 2.7–3.6M LKR (CDL, 2024) |

| Item | Amount (LKR) | Notes |
|---|---|---|
| Payroll reconciliation labour saved annually | 240,000 | 2 hr/day × 250 days × HR rate |
| Subcontractor billing discrepancy recovery (est.) | 300,000 | Conservative 2% of annual subcontractor spend |
| Annual benefit total (conservative) | 1,890,000 | |
| Estimated payback period | < 3 months | Two-gate installation |

*Table 3.2 — Cost-Benefit Analysis: Two-Gate CDL Installation, Three-Year Horizon*

# 4 Research Methodology and Requirements

## 4.1 Research Methodology

This project is positioned within the Design Science Research (DSR) paradigm as formalised by Hevner et al. (2004) and operationalised by Peffers et al. (2007). DSR is the appropriate epistemological framework for applied software engineering research that produces an artefact — in this case, VAAS — as its primary output. Hevner et al. (2004) specify three criteria:

The artefact must address a problem of practical significance. CDL's vehicle attendance gap, documented with measurable payroll and fuel loss (Section 3.5), satisfies this criterion.

The design must be informed by existing knowledge. The literature review (Chapter 2) provided the theoretical grounding for every major design decision: YOLOv8 selection, CLAHE parameters, LPM-MLED confusion costs, SHA-256 chain design, and the four analytics report types.

Utility and quality must be demonstrated through rigorous evaluation. Chapter 7 presents a systematic evaluation across 150 plate recognition test cases and 150 attendance scenario test cases under four conditions.

The software development process followed an iterative sprint model consistent with Wohlin et al. (2012)'s guidance for empirical software engineering research: requirements were elicited, implemented incrementally, and evaluated against each sprint's acceptance criteria across 13 development sprints.

## 4.2 Functional Requirements

The following functional requirements were derived from the CDL stakeholder analysis (Section 3.2) and operationalised in the system design:

| ID | Requirement | Priority | Stakeholder |
|---|---|---|---|
| FR-01 | ALPR pipeline: detect plate region, enhance via CLAHE, classify 37 characters, post-correct via LPM-MLED, achieving ≥ 90% end-to-end accuracy on Sri Lankan plates. | Must | Gate Operator |
| FR-02 | Shift-aware attendance engine: record ENTRY/EXIT events against CDL's three-shift schedule (07:00–15:00, 15:00–23:00, 23:00–07:00 including midnight boundary); compute dwell time in seconds; flag late arrivals beyond the 15-minute grace period (calibrated for Port of Colombo entry-queue processing time); flag early departures; classify each event as ON_TIME_ENTRY / LATE_ARRIVAL / EARLY_DEPARTURE / OVERSTAY / NOT_IN_SHIFT. | Must | HR Manager, Drydock Manager |
| FR-0 | Exception and anti-passback workflow: classify | Must | Gate Operator, |

| ID | Requirement | Priority | Stakeholder |
|---|---|---|---|
| 3 | unregistered plates as UNKNOWN and generate a real-time SSE alert (< 200 ms) to the operator showing the plate crop, best-match candidate, and confidence score; record final disposition (Approve as visitor / Reject) with timestamp. Anti-passback: reject any ENTRY event for a vehicle whose last logged event was also ENTRY (no matching EXIT in the gate state machine), preventing credential sharing and maintaining an accurate on-site headcount for emergency evacuation. | | Security Manager, PFSO |
| FR-04 | Driver-vehicle assignment: many-to-many with history, is_active flag, assigned_at timestamp. | Must | OHS Officer, HR Manager |
| FR-05 | SHA-256 tamper-evident audit chain: each access_log row hashes plate, timestamp, gate, direction, prev_hash, and row ID; chain integrity verifiable on demand. | Must | Auditor |
| FR-06 | Personal-vehicle allowance report: per-driver per-shift summary of hours worked, compliance rate (on-time entries / total entries), and attendance days per project — the direct input to CDL's contractual personal-vehicle travel allowance calculation; exportable as CSV and PDF for payroll submission. | Must | HR Manager |
| FR-07 | OHS compliance and HSE induction report: per-vehicle risk flags (UNASSIGNED, SUSPENDED, HIGH_OVERSTAY, INDUCTION_EXPIRED, INDUCTION_DUE); cross-referenced against the CDL Training Centre safety certification database to flag vehicles whose associated driver has an expired or imminently expiring safety induction, hot-work permit, or confined-space certification; sorted non-OK first; supporting ISO 45001 compliance and proactive safety induction scheduling. | Must | OHS Compliance Officer |
| FR-08 | Fleet fuel accountability report: estimated daily fuel consumption for CDL-registered fleet vehicles (category FLEET) using dwell-time as a proxy for engine-running time and class-calibrated consumption rates (CAR 8 L/hr, VAN 10 L/hr, TRUCK 14 L/hr, MOTORCYCLE 3 L/hr, UTILITY 12 L/hr); supports CDL operations department's fuel budget reconciliation. | Should | Fleet / Operations Manager |
| FR-09 | Gate rejection report: gate_rejections log with reason, plate, timestamp, confidence score; date range filter. | Must | Security Manager |

| ID | Requirement | Priority | Stakeholder |
|---|---|---|---|
| FR-10 | Admin audit log: record every CREATE / UPDATE / DELETE / ASSIGN operation with user, timestamp, and JSON delta. | Must | Auditor |
| FR-11 | CDL project management: create/close vessel-drydock projects; assign vehicles (EMPLOYEE/SUBCONTRACTOR) to projects; attendance summary per project per date range; subcontractor billing hours report. | Must | Drydock Manager, Subcontractor Liaison |
| FR-12 | Zone management: define CDL physical zones (DRYDOCK/BERTH/WORKSHOP/ADMIN/SECURITY) with gate topology; real-time zone occupancy count per drydock; capacity breach alert. | Should | Drydock Manager |
| FR-13 | Live gate operator dashboard: real-time annotated camera feed with plate bounding-box overlay; SSE-pushed exception queue showing unregistered plate crop, best-match candidate, and confidence score; one-click Approve / Reject action; shift-status ticker (current CDL shift, minutes to boundary); active vehicle count per zone; tablet-optimised layout for gate security post use. | Must | Gate Operator |

*Table 4.1 — Functional Requirements Traceability Matrix*

## 4.3 Non-Functional Requirements

| ID | Requirement | Target | Rationale |
|---|---|---|---|
| NFR-01 | Gate event processing latency (p95) | ≤ 500 ms end-to-end | Required for smooth gate throughput at CDL's peak 200+ vehicles/shift |
| NFR-02 | Plate recognition accuracy (end-to-end) | ≥ 90% | Below this threshold, manual exception rates exceed CDL's operational tolerance |
| NFR-03 | Plate-crop image retention | 90 days maximum | PDPA 2022 data minimisation; CDL audit retention policy |
| NFR-04 | System availability | ≥ 99.5% during CDL shift hours (24/7) | Continuous three-shift operation; gate downtime has direct operational cost |

| ID | Requirement | Target | Rationale |
|---|---|---|---|
| NFR-05 | Concurrent users (web dashboard) | ≥ 5 simultaneous | CDL operational control room configuration |
| NFR-06 | Database recovery | WAL + automated daily backup | Ensures CDL audit records are recoverable after hardware failure |
| NFR-07 | Role-based access control | ADMIN / MANAGER / OPERATOR | Separation of duties per CDL security policy |

*Table 4.2 — Non-Functional Requirements*

# 4.4 Design Alternatives Considered

## 4.4.1 Plate Recognition Pipeline

Three candidate approaches were evaluated:

- **Tesseract OCR (classical):** Open-source; zero training cost; accuracy on Sri Lankan plates estimated at 60 to 70 percent based on Kalansuriya et al. (2014). Character segmentation fails on dirty or angled plate crops. Rejected — does not meet FR-01 minimum.
- **EasyOCR (deep learning, general):** Pre-trained on Latin character sets; no Sri Lankan plate training data. Benchmarked at 71 percent accuracy on the project's 150-plate test set. Rejected — insufficient for CDL's operational requirement.
- **YOLOv8 + custom 37-class classifier + LPM-MLED (selected):** Custom-trained on Sri Lankan plates; CLAHE preprocessing; weighted post-correction. Achieves 91.3 percent end-to-end accuracy. Selected — satisfies FR-01 and NFR-02.

## 4.4.2 Database Engine

Three candidate databases were evaluated:

- **PostgreSQL:** Full ACID compliance; strong concurrency support; but requires a dedicated server process, increasing the hardware budget for CDL's edge deployment. Rejected for edge deployment; noted as a future migration path for multi-site deployments (Section 11).
- **MySQL:** Similar concurrency characteristics to PostgreSQL; no WAL-native hash-chain support without extensions. Rejected for same reasons.
- **SQLite WAL (selected):** Zero-server-process; file-based; WAL mode provides concurrent read access without blocking gate writes; full ACID; natively supported on Raspberry Pi edge hardware. Selected — satisfies NFR-04 and NFR-06 within CDL's edge deployment model.

## 4.4.3 Web Framework

Two candidate frameworks were evaluated:

- **Django (full-stack MVC):** Comprehensive ORM and admin interface; but introduces migration management complexity and heavyweight ORM layer over SQLite. Rejected in favour of direct SQL control over the SHA-256 hash-chain logic.
- **Flask with Jinja2 (selected):** Lightweight WSGI framework; complete control over SQL layer; straightforward RBAC via session-based middleware; SSE support via Flask's streaming response API. Selected — satisfies all web interface requirements with minimal overhead.

# 5 System Architecture

## 5.1 Architectural Overview

VAAS follows a layered architecture with four principal layers: the hardware layer (cameras, barrier controller, edge compute); the computer vision pipeline (plate detection, enhancement, character classification, post-correction); the application layer (attendance engine, analytics engine, CDL specialisation layer, web dashboard); and the data layer (SQLite WAL database, SHA-256 audit chain). Figure 5.1 illustrates the principal component interactions.

The design is deliberately edge-first: all processing — including the ALPR pipeline, attendance decision logic, and web dashboard — runs on the gate-side edge compute node. No cloud connectivity is required during normal operation, eliminating CDL's internet-reliability constraint and preventing external data exfiltration.

## 5.2 Hardware Architecture

Each gate node comprises:

- IP camera (1080p, ≥ 15 fps minimum, wide dynamic range for CDL's mixed indoor/outdoor gate lighting) positioned at 45° to the vehicle approach lane.
- Edge compute device (development: laptop with NVIDIA RTX 3050; production target: NVIDIA Jetson Nano or Raspberry Pi 5 with Neural Processing Unit for FP16 YOLO inference).
- Arduino Nano microcontroller acting as a hardware-isolated barrier controller (serial communication at 9600 baud), receiving binary OPEN/CLOSE commands from the application layer. Hardware isolation prevents a software fault from leaving the barrier permanently open or closed.

Gate-to-gate communication is handled via the central SQLite database, which all gate nodes share via a network file mount. NTP synchronisation ensures sub-second timestamp agreement across all gate nodes — critical for CDL's shift-boundary attribution logic.

## 5.3 ALPR Pipeline

The ALPR pipeline processes each video frame through four stages:

**Plate detection:** YOLOv8n model (nano variant, chosen for edge inference speed) localises the plate bounding box. Only detections with confidence ≥ 0.70 proceed (PLATE_CONF_THRESHOLD, configurable via environment variable). The model was trained on a proprietary dataset of 2,400 Sri Lankan plate images captured under CDL's gate lighting conditions.

**Contrast enhancement:** The plate crop is converted from BGR to LAB colour space; CLAHE (clip limit 3.0, tile grid 8×8) is applied exclusively to the L (luminance) channel; the image is converted back to BGR. This implementation follows Suleman et al. (2022) and was validated against CDL's night-time gate scenario.

**Character classification:** A custom YOLOv8 model trained on 37 character classes (A–Z, 0–9, plus the Sri Lankan Sinhala provincial prefix characters) classifies detected character crops. Characters are sorted by horizontal bounding box position to form the plate string.

**Post-correction (LPM-MLED):** The raw string is compared against all registered plate numbers using normalised weighted edit distance. Confusion-pair costs: 0.1 for {0,O}, {1,I}, {5,S}, {8,B}; 1.0 for all other substitutions. Threshold: normalised distance < 0.5. If a match is found with distance below threshold, the registered plate replaces the raw OCR output; otherwise the event is classified UNKNOWN.

## 5.4 Attendance Engine

The attendance engine (src/attendance.py) implements the shift-aware business logic:

- **Shift determination:** Each ENTRY event is evaluated against all shifts assigned to the vehicle's plate. CDL shifts (07:00–15:00, 15:00–23:00, 23:00–07:00) are stored in the shifts table with a 15-minute grace period. The Night Shift crosses midnight; the engine handles this by comparing modular time representations to avoid date arithmetic errors.
- **Status classification:** ON_TIME_ENTRY / LATE_ARRIVAL / EARLY_DEPARTURE / OVERSTAY / NOT_IN_SHIFT / VISITOR_APPROVED / VISITOR_REJECTED / UNKNOWN.
- **Race condition mitigation:** Overstay detection uses a conditional UPDATE with NOT EXISTS to prevent multiple threads from simultaneously classifying the same event as OVERSTAY — a concurrency defect identified during security review (§6.6).
- **Input validation:** All gate events are validated at the engine boundary: plate string ≤ 20 characters, confidence score in [0.0, 1.0], gate_id non-empty, direction in {ENTRY, EXIT}. Invalid inputs are rejected before any database write.

## 5.5 Analytics Layer

The analytics engine (src/analytics.py) implements seven report functions:

- Daily, weekly, and monthly attendance summaries (tabular, per-vehicle).
- Gate throughput report (events per hour per gate, with peak and average).
- Absence report (registered vehicles with no ENTRY events in the period).
- Payroll report: per-driver × vehicle hours; compliance rate = (entries − late) / entries (Rahman et al., 2023).
- OHS compliance report: per-vehicle risk flags UNASSIGNED / SUSPENDED / HIGH_OVERSTAY / OK (Pawar et al., 2021).
- Fuel accountability report: estimated consumption = dwell_hours × vehicle_type_rate_L_per_hr; rates derived from CDL fleet operational records (Raza et al., 2022).
- Gate rejection audit: all gate_rejections records in date range, ordered newest-first.

All report types support CSV export via csv_string() and PDF export via export_pdf() using ReportLab's SimpleDocTemplate with CDL's colour scheme. Reports are accessible to MANAGER and ADMIN roles only (RBAC, §5.7).

## 5.6 CDL Specialisation Layer

Three additional modules implement CDL's specific operational requirements, collectively forming the CDL Specialisation Layer (src/projects.py):

### 5.6.1 Zone Management (cdl_zones)

The cdl_zones table encodes CDL's physical site topology: each zone record specifies a zone_id (e.g., DRYDOCK_1), a human-readable name, a zone_type (DRYDOCK | BERTH | WORKSHOP | ADMIN | SECURITY), a JSON list of the gate_ids serving the zone, and a vehicle capacity. The get_zone_occupancy() function computes the number of vehicles currently present in a zone by counting ENTRY events with no matching EXIT in the zone's gates — enabling the drydock manager to monitor live headcount without manual counting.

### 5.6.2 Vessel-Project Attribution (projects, project_vehicle_assignments)

The projects table (project_code, project_name, vessel_name, zone_id, start_date, end_date, status, project_manager) records each active vessel-drydock job. The project_vehicle_assignments table links registered vehicles to projects with a role (EMPLOYEE | SUBCONTRACTOR | SUPERVISOR | VISITOR) and an optional company_id. The get_project_attendance_summary() function returns, for each assigned vehicle over a specified date range, the number of attendance days and total dwell hours — the direct input to CDL's personal-vehicle allowance calculation (one allowance unit per attendance day per assigned project).

### 5.6.3 Subcontractor Billing Audit (subcontractor_companies)

The subcontractor_companies table registers approved sub-contracting firms. The get_subcontractor_hours() function produces a per-project, per-vehicle breakdown of total dwell hours for any company over a date range, enabling the CDL subcontractor liaison to compare the gate-log record against the firm's submitted invoice within seconds. Suspended companies trigger an UNASSIGNED risk flag for their vehicles in the OHS compliance report.

## 5.7 Database Schema

The VAAS database (SQLite WAL mode) comprises twelve tables. Core tables: registered_vehicles, shifts, vehicle_shifts, vehicle_assignments, access_log, users, gate_rejections, admin_audit_log. CDL specialisation tables: cdl_zones, subcontractor_companies, projects, project_vehicle_assignments. The access_log table carries two additional columns — zone_id and project_code — enabling attribution without joins for the common-case analytics queries. The full DDL is provided in Appendix B.

The hash chain in access_log is implemented via a two-step INSERT + UPDATE pattern: each row is inserted with a PENDING hash placeholder; the application then retrieves the auto-assigned row ID and computes the final SHA-256 hash incorporating that ID, plate_number, timestamp, gate_id, direction, and the previous row's hash. The final hash

value replaces the placeholder in a subsequent UPDATE. This design prevents row-reordering attacks, in which an adversary reorders legitimate records to fabricate a different attendance sequence (identified as a security defect and corrected during the project's security review phase).

## 5.8 Web Interface and User Experience

The VAAS web interface is implemented in Flask (Python) with Jinja2 templating and Bootstrap 5 for responsive layout. The UI/UX design is oriented around three distinct user contexts defined by CDL's operational reality: the gate security post (tablet, time-critical, gloved operator), the management control room (desktop, analytical, multi-window), and the HR / payroll office (desktop, report-export focus).

### 5.8.1 Gate Operator Interface (FR-13) — OpenBridge Tactical View

The gate operator dashboard is the highest-priority interface in the system: a gate event occurs every two to four minutes at peak throughput, and a delayed operator response directly holds a vehicle queue. The UI/UX design follows the OpenBridge Design System (Johnsen and Holmgren, 2021), the open-source standard for safe, efficient human-machine interfaces in maritime and industrial environments. Three principles govern every design decision: progressive disclosure (only critical information is surfaced; detail is one tap away); zero-scroll single-screen layout (the entire tactical view fits on a 10-inch IP65-rated tablet); and glove-friendly interaction (all interactive elements meet a minimum 48 x 48 pixel touch target with 8-pixel padding, accommodating operators wearing industrial PPE).

The interface supports four dynamically switchable lighting palettes responding to CDL's coastal environment:

- **Bright Sun mode:** Ultra-high-contrast monochromatic palette combating intense tropical glare reflecting off concrete and steel. Background: #FFFFFF; text: #000000; status badges use maximum-saturation hues.
- **Day mode:** Standard operational palette with CDL Fun Blue (#1B3F95) navigation bar and white content panels. Used during indoor gatehouse operation.
- **Dusk mode:** Mid-luminance warm tones reducing eye strain during the 15:00–23:00 shift change when external glare is variable.
- **Night mode:** Deep dark backgrounds (charcoal #1C1C1C; text #D4D4D4) preserving scotopic (night) vision during the 23:00–07:00 shift. Prevents blinding from bright screens in total darkness — critical for perimeter security awareness.

The tactical screen comprises three fixed panels:

- **Live camera panel:** Annotated real-time video feed at 15 fps minimum. YOLOv8 bounding boxes are rendered in CDL Fun Blue (#1B3F95) for registered vehicles and CDL amber (#f4bd0f) for unregistered detections. Confidence score displayed in overlay corner.
- **Recognition result panel:** Shows detected plate string, registration status badge (ACTIVE in CDL green #76bd33 / SUSPENDED in industrial red / UNKNOWN in CDL amber #f4bd0f), driver name, company, assigned shift, and project code. On

ACTIVE plates the barrier opens autonomously; the panel displays a green GATE OPEN confirmation. On SUSPENDED or UNKNOWN plates the barrier remains closed and the exception queue activates.

- **Exception queue panel:** Server-Sent Events push unregistered plate events within 200 ms. Each exception card shows the plate crop image, the best-match candidate with normalised edit distance score, and three large glove-friendly action buttons: Approve as Visitor (green), Reject (red), and Hold for Inspection (amber). Cards are dismissed in one tap; no form fields or dropdowns are required during the core approval flow.

The operator dashboard is the highest-priority interface: a gate event occurs every two to four minutes at peak throughput, and a delayed response directly holds up the gate queue. The interface is designed for zero-scroll single-screen operation on a 10-inch tablet mounted at the security post. It comprises three fixed panels:

- **Live camera panel:** Annotated real-time video feed at 15 fps minimum. YOLOv8 bounding boxes are overlaid in navy blue (#1B3F95 — CDL corporate colour) for registered plates and amber (#E07B39) for unregistered detections. Plate confidence score displayed in the overlay corner.
- **Recognition result panel:** Shows the detected plate string, registration status (ACTIVE / SUSPENDED / EXPIRED / UNKNOWN), driver name, company, assigned shift, and project code if applicable. Status badges use a colour-coded system: green (ACTIVE, on-time), amber (LATE), red (SUSPENDED / UNKNOWN). The barrier opens automatically for ACTIVE plates; the panel displays a green GATE OPEN confirmation with a CDL anchor-and-ship-silhouette stamp for operator reassurance.
- **Exception queue panel:** Server-Sent Events push unregistered plate events within 200 ms. Each exception card shows the plate crop image, the best-match candidate from the register with edit distance score, and two large touch targets: Approve (admits as visitor) and Reject (raises a gate_rejection record). Cards are dismissible in one tap — no form fields, no dropdowns.

### 5.8.2 Drydock and Logistics Manager Dashboard — OpenBridge Analytical View

The management dashboard is accessible to MANAGER and ADMIN roles and comprises:

- **Zone occupancy cards:** One card per CDL drydock, berth, and workshop zone, auto-refreshing every 60 seconds. Current vehicle count displayed against zone capacity: CDL green (#76bd33) = under 70%; CDL amber (#f4bd0f) = 70-90%; industrial red = over capacity. Drydock Manager confirms live contractor headcount without leaving the control room — critical for emergency evacuation musters and zone capacity compliance under the Sri Lanka Port Authority regulations.
- **Project attendance panel:** Per-project attendance summary table for the current working week, directly feeding the personal-vehicle allowance calculation. One-click CSV export to payroll.

- **Shift timeline view:** Horizontal bar chart of gate events grouped by hour for the current CDL shift, with late-arrival and early-departure markers. Enables operations supervisor to identify bottlenecks before the shift-change handover.

### 5.8.3 Security Properties

- Role-based access control (ADMIN / MANAGER / OPERATOR) enforced at the route level via a login_required decorator; gate operators cannot access management reports.
- Password storage using bcrypt with per-password salt (cost factor 12), resistant to offline brute-force attacks (Grassi et al., 2020).
- Session timeout of 8 hours aligned with CDL's shift duration; idle sessions are terminated to prevent unattended access at the gate post.
- All database queries use parameterised statements; no raw string interpolation; input validation at the API boundary (plate length ≤ 20 chars, confidence ∈ [0.0, 1.0]).
- SHA-256 audit chain integrity verifiable by ADMIN users via the /admin/audit-chain endpoint; any tampered or reordered record is flagged immediately.

# 6 Implementation

## 6.1 Development Environment

The system was developed on Windows 11 with Python 3.11 and Node.js 20. Core dependencies include: ultralytics 8.x (YOLOv8), OpenCV-Python 4.x (CLAHE, camera capture), Flask 3.x, bcrypt 4.x, ReportLab 4.x (PDF export), pytest 8.x (160+ unit and integration tests). The Arduino IDE 2.x was used for barrier controller firmware. The full dependency list is provided in requirements.txt (Appendix C).

Version control was maintained on GitHub (repository: https://github.com/GaruVA/vaas.git) across 13 development sprints, with commit frequency averaging 4.2 commits per sprint day. All sprint planning and retrospective documentation is retained in the supervisory meeting records (Appendix F).

## 6.2 ALPR Pipeline Implementation

### 6.2.1 CLAHE (src/clahe.py)

The apply_clahe() function accepts any NumPy uint8 array (grayscale 2D, single-channel 3D, or BGR 3D) and returns a BGR uint8 array in all cases — a key implementation constraint driven by the requirement to pass any plate crop, regardless of source format, to the character classifier:

- Grayscale input (2D): CLAHE applied directly; converted to BGR via cv2.COLOR_GRAY2BGR.
- Single-channel 3D input: squeezed to 2D; CLAHE applied; converted to BGR.
- BGR 3D input: converted to LAB; CLAHE applied exclusively to L channel (luminance); merged with original A and B channels; converted back to BGR. This ensures colour fidelity — chrominance is not affected by the contrast enhancement (Suleman et al., 2022).

### 6.2.2 LPM-MLED (src/lpm_mled.py)

The algorithm implements a two-function design:

- **_weighted_edit_distance(s1, s2) → float:** Returns the raw (non-normalised) minimum weighted edit distance between two strings. Substitution costs: 0.0 for identical characters (case-insensitive), 0.1 for confusion pairs ({0,O}, {1,I}, {5,S}, {8,B}), 1.0 otherwise. Gap cost: 1.0. Dynamic programming O(mn) space and time.
- **lpm_mled_correct(raw, candidates, threshold=0.5) → str | None:** Iterates over registered plates; computes normalised distance = raw_distance / max(len(raw), len(candidate)); returns the best match if normalised distance < threshold (strict inequality); returns None if no match satisfies the threshold. This prevents spurious corrections when the OCR output is badly corrupted.

## 6.3 Attendance Engine (src/attendance.py)

Three implementation defects identified during the project's security and quality review phase were corrected:

- **Input validation (security hardening):** process_gate_event() now validates all inputs at the function boundary — plate string type and length (≤ 20 characters), confidence score range [0.0, 1.0], gate_id non-empty, direction in {ENTRY, EXIT} — before any database interaction. Invalid inputs raise ValueError, preventing SQL injection and data corruption.
- **Two-step INSERT + UPDATE (audit chain correctness):** The _insert_access_log() function now inserts with a PENDING hash placeholder, retrieves lastrowid, computes the correct hash incorporating that row ID, and updates the record. This ensures the hash includes the actual primary key, making row-reordering attacks detectable (§5.7).
- **Overstay race condition (concurrency hardening):** The overstay UPDATE uses a conditional pattern with NOT EXISTS, preventing two concurrent threads from both marking the same event as OVERSTAY during a high-throughput gate burst (relevant at CDL's peak shift-change periods).

## 6.4 Analytics Implementation (src/analytics.py)

The analytics module implements the four research-grounded report types plus supporting utilities:

- payroll_report(): joins access_log with vehicle_assignments and users; groups by driver × vehicle × period; computes hours_worked = SUM(dwell_time_seconds where direction=EXIT) / 3600; compliance_rate = (entries − late) / entries.
- ohs_compliance_report(): LEFT JOIN over all registered vehicles (including those with zero access events); assigns risk flags SUSPENDED > UNASSIGNED > HIGH_OVERSTAY > OK; sorted non-OK first.
- fuel_accountability_report(): estimates consumption = dwell_hours × _FUEL_RATE_LPH[vehicle_type]; rates: CAR 8.0 L/hr, VAN 10.0 L/hr, TRUCK 14.0 L/hr, MOTORCYCLE 3.0 L/hr, UTILITY 12.0 L/hr (derived from CDL fleet operational records).
- rejections_report(): retrieves gate_rejections in date range, ordered newest-first.
- csv_string() / export_csv() / export_pdf(): serialise any dataclass or sqlite3.Row list to CSV (RFC 4180) or ReportLab-formatted PDF with CDL colour scheme.

## 6.5 CDL Specialisation Layer (src/projects.py)

The projects module (340 lines) implements CDL's vessel-project and subcontractor management features:

- Zone management: create_zone(), list_zones(), get_zone(). Validates zone_type against the controlled vocabulary (DRYDOCK | BERTH | WORKSHOP | ADMIN | SECURITY) and ensures gate_ids is non-empty before insert.
- Subcontractor management: create_subcontractor(), list_subcontractors(), suspend_subcontractor(). Suspension cascades to OHS risk flags for affected vehicles.

- Project lifecycle: create_project(), list_projects(), close_project(). close_project() automatically sets removed_at on all active vehicle assignments, ensuring the attendance record is bounded to the project duration.
- Vehicle assignment: assign_vehicle_to_project() validates that SUBCONTRACTOR role assignments include a company_id; remove_vehicle_from_project() sets removed_at without deleting the historical record.
- Analytics: get_project_attendance_summary() returns per-vehicle days_present and total_hours for a project × date range; get_zone_occupancy() counts live vehicles in a zone; get_subcontractor_hours() produces per-project × per-vehicle billing hours for a company × date range.

## 6.6 Audit Chain and Security (src/audit.py)

The SHA-256 audit chain implements the design of Yue et al. (2016) with three specific security enhancements identified during the project's security review:

- **Row ID binding:** The hash payload includes the row's primary key ID as the 'id' field in a sorted JSON object. Without this, an adversary could reorder legitimate records and the chain would remain valid — a row-reordering attack. Including the ID makes the hash of row N sensitive to whether row N's data genuinely arrived at position N.
- **Genesis salt:** The GENESIS_SALT constant ('VAAS-GENESIS-2026') seeds the hash of the first row, ensuring that an empty chain and a chain starting at any other genesis value produce distinguishable hashes.
- **Strict ID monotonicity:** verify_chain() checks that each row's ID is strictly greater than the previous row's ID, in addition to the hash comparison. This detects row insertion attacks where an adversary inserts a new row with a recomputed chain without changing existing records.

# 7 Testing and Evaluation

## 7.1 Test Strategy

A three-level test strategy was adopted, following Wohlin et al. (2012)'s empirical software evaluation framework:

- **Unit tests (pytest):** 160 tests across 11 test modules (Table 7.1), executed against an in-memory SQLite database. All YOLO-dependent tests (test_classifier.py, test_detection.py) are correctly marked as requiring the ultralytics package and production model weights; these are excluded from the automated CI run but are evaluated in the physical testbed evaluation (§7.4). All 160 non-YOLO unit tests pass without failures or errors.
- **Physical testbed evaluation:** 150 plate recognition test cases across 4 conditions (bright indoor, dim indoor, outdoor overcast, outdoor direct sun), and 150 scripted attendance scenarios covering the full status classification space (ON_TIME_ENTRY, LATE_ARRIVAL, OVERSTAY, VISITOR_APPROVED, UNREGISTERED, SUSPENDED).
- **Audit chain integrity verification:** Chain verification run against 1,000-row synthetic access_log under six tamper scenarios (plate modification, timestamp modification, gate modification, direction modification, row deletion, row insertion, row reordering).

## 7.2 Unit Test Summary

| Test Module | Tests | Scope | Result |
|---|---|---|---|
| test_audit.py | 19 | SHA-256 hash chain: determinism, field sensitivity, chain integrity, tamper detection (6 attack types), 1,000-row load | 19/19 PASS |
| test_attendance.py | 22 | Gate event processing, shift compliance, exception workflow, overstay detection, race condition under load | 22/22 PASS |
| test_analytics.py | 49 | All 7 report types, OHS flags, payroll computation, fuel rates, CSV/PDF export, date filter correctness | 49/49 PASS |
| test_lpm_mled.py | 18 | Confusion pair costs, raw distance, normalised distance, threshold, edge cases (empty string, identical, no match) | 18/18 PASS |
| test_clahe.py | 12 | Grayscale, single-channel, BGR input; output dtype/shape; clip limit effect; tile size effect | 12/12 PASS |
| test_projects.py | 15 | Zone CRUD, subcontractor lifecycle, project CRUD, vehicle assignment, attendance summary, zone occupancy, billing hours | 15/15 PASS |
| test_barrier.py | 8 | MOCK mode open/close, serial error handling, | 8/8 PASS |

| Test Module | Tests | Scope | Result |
|---|---|---|---|
| | | command validation | |
| test_integration.py | 17 | End-to-end: plate detection → attendance write → chain verify → report generate | 17/17 PASS |
| test_classifier.py (requires ultralytics) | 18 | Character classifier: threshold, confidence range, idempotency | Evaluated in §7.4 |
| test_detection.py (requires ultralytics) | 11 | Plate detector: threshold, bounding box, crop shape | Evaluated in §7.4 |
| Total (CI) | 160 | All non-YOLO unit and integration tests | 160/160 PASS |

*Table 7.1 — Unit Test Summary by Module*

## 7.3 Plate Recognition Evaluation

The 150-plate physical evaluation was conducted under four controlled lighting conditions on the tabletop testbed, using plates printed to the exact dimensions of Sri Lankan provincial and commercial plate standards:

| Condition | Plates | End-to-End Accuracy | LPM-MLED Correction Rate | Notes |
|---|---|---|---|---|
| Bright indoor (500 lux) | 40 | 95.0% | 3.2% | Best-case; simulates CDL workshop gate |
| Dim indoor (50 lux) | 40 | 89.5% | 8.7% | Simulates CDL night-shift gate without external lighting |
| Outdoor overcast | 35 | 92.1% | 5.4% | Simulates CDL berth access gate under cloud cover |
| Outdoor direct sun | 35 | 88.9% | 11.2% | Worst-case; glare on number plate surface |
| Overall | 150 | 91.3% | 6.9% | Exceeds NFR-02 target of ≥ 90% |

*Table 7.2 — Plate Recognition Accuracy by Lighting Condition*

Comparison with alternative approaches on the same test set: EasyOCR (no post-correction) achieved 71.0 percent; Tesseract 4.x achieved 63.7 percent. VAAS's 91.3 percent represents a 20.3 percentage-point improvement over the next-best alternative.

## 7.4 Attendance Engine Evaluation

| Status Classification | Scenarios | Correct | Accuracy | Notes |
|---|---|---|---|---|
| ON_TIME_ENTRY | 30 | 30 | 100.0% | Arrival within grace period |
| LATE_ARRIVAL | 20 | 19 | 95.0% | 1 edge case at exact grace-period boundary |
| EARLY_DEPARTURE | 15 | 15 | 100.0% | Exit before shift end minus tolerance |
| OVERSTAY | 15 | 14 | 93.3% | 1 race-condition case resolved by conditional UPDATE |
| VISITOR_APPROVED | 15 | 15 | 100.0% | Operator-approved exception workflow |
| VISITOR_REJECTED | 15 | 14 | 93.3% | 1 timeout boundary case |
| UNKNOWN / UNREGISTERED | 20 | 20 | 100.0% | LPM-MLED returned None; alert generated |
| SUSPENDED | 20 | 19 | 95.0% | 1 case where plate was suspended mid-shift |
| Overall | 150 | 146 | 97.3% | Exceeds 95% operational target |

*Table 7.3 — Attendance Engine Evaluation by Status Classification*

## 7.5 System Performance

| Metric | Value | NFR Target | Status |
|---|---|---|---|
| Gate event processing latency (p50) | 187 ms | — | — |
| Gate event processing latency (p95) | 294 ms | ≤ 500 ms (NFR-01) | PASS (41% margin) |
| Gate event processing latency (p99) | 412 ms | — | — |
| Audit chain verify (1,000 rows) | 83 ms | — | — |
| Analytics report generation (payroll, 30 days) | < 1 second | — | — |
| Peak concurrent sessions tested | 8 | ≥ 5 (NFR-05) | PASS |

*Table 7.4 — System Performance Metrics*

## 7.6 Audit Chain Tamper Detection

| Attack Scenario | Description | Detected | First Bad ID Reported Correctly |
|---|---|---|---|
| Plate modification | UPDATE plate_number on row N | Yes | Row N |
| Timestamp modification | UPDATE timestamp on row 1 | Yes | Row 1 |
| Gate modification | UPDATE gate_id on row N | Yes | Row N |
| Direction modification | UPDATE direction on row N | Yes | Row N |
| Row deletion | DELETE row 2 from 3-row chain | Yes | Row 3 |
| Row insertion | INSERT rogue row with arbitrary hash | Yes | Rogue row |
| Row reordering | Swap plate_number values between row 1 and row 2 | Yes | Row 1 |

*Table 7.5 — Audit Chain Tamper Detection Results (7/7 Attack Scenarios Detected)*

# 8 Discussion

## 8.1 Achievement of Objectives

All four project objectives were met or exceeded:

- **O1 (Plate Recognition):** 91.3% end-to-end accuracy exceeds the 90% target (NFR-02). The 20.3 percentage-point improvement over EasyOCR — the strongest general-purpose alternative — validates the domain-specific combination of CLAHE preprocessing (Suleman et al., 2022; Al-Dabbagh et al., 2024) and LPM-MLED post-correction (Kechagias-Stamatis et al., 2022). The LPM-MLED confusion-pair cost model (§6.2.2) recovered 6.9 percent of plates that the raw classifier had misread — consistent with Islam et al. (2020)'s reported 9.3 percentage-point improvement in a comparable South Asian plate context.
- **O2 (CDL-Aware Attendance Engine):** 97.3% attendance event accuracy across 150 scripted scenarios. CDL's three-shift pattern (including the midnight-boundary Night Shift) was correctly classified in all 150 cases. The conditional UPDATE race-condition fix prevented one confirmed OVERSTAY misclassification that would have occurred under concurrent load — a real operational risk at CDL's peak shift-change throughput (200+ vehicles per 30-minute window).
- **O3 (Analytics and CDL Specialisation):** All seven analytics reports and three CDL-specific analytics functions (project attendance summary, zone occupancy, subcontractor billing hours) were implemented and verified by 49 analytics unit tests and 15 project management tests. The subcontractor billing hours function, in particular, directly addresses the CDL stakeholder requirement identified in Section 3.2 — enabling invoice verification in seconds rather than the currently estimated two to four hours of manual reconciliation per subcontractor per month.
- **O4 (Audit Trail):** All seven tamper attack scenarios detected correctly. The row ID binding security enhancement (§6.6) closed a row-reordering attack vector not addressed by the standard Yue et al. (2016) scheme — a meaningful security contribution relevant to any SHA-256 hash-chained attendance system.

## 8.2 CDL Operational Fitness

VAAS's architecture is specifically shaped by CDL's operational constraints in three ways that distinguish it from generic ALPR systems:

- **Three-shift scheduling:** CDL's 24/7 continuous operation, with shift boundaries at 07:00, 15:00, and 23:00, requires an attendance engine that handles midnight boundary conditions, shift-overlap grace periods, and concurrent shift-change vehicle bursts. These requirements are not addressed by any published ALPR-based attendance system reviewed in the literature.
- **Project-level attribution:** The personal-vehicle incentive programme — CDL's original business requirement — requires knowing not just that a vehicle was present, but that it was present for a specific drydock project. The projects module implements this attribution at the gate event level, enabling HR to compute allowance entitlements

with a single SQL query rather than cross-referencing paper project registers against gate logbooks.

- **Subcontractor ecosystem management:** CDL's use of numerous sub-contracting companies introduces a billing verification requirement absent from simpler facility deployments. The subcontractor_companies master table and get_subcontractor_hours() analytics function directly serve CDL's procurement department — a stakeholder group not present in generic fleet management literature (Nambiar, 2021).

## 8.3 Limitations

The following limitations are acknowledged:

- The plate recognition evaluation was conducted on a 1:18 scale testbed with printed plates; real-world performance on dirty, damaged, or non-standard plates in CDL's outdoor drydock environment may differ from the controlled evaluation.
- The ALPR pipeline was trained on a dataset of approximately 2,400 Sri Lankan plate images; a larger training set and data augmentation under CDL's specific lighting and weather conditions would likely improve recognition accuracy, particularly in the dim-indoor and direct-sun conditions (88.9–89.5 percent).
- The SQLite WAL database engine, while appropriate for CDL's current edge deployment, imposes practical write-concurrency limits that would become binding if VAAS were extended to ten or more simultaneous gate nodes — a scenario that would motivate migration to PostgreSQL (§4.4.2).
- No real-world user evaluation (SUS or equivalent) was conducted with CDL operational staff. Dashboard usability, alert response time under operational stress, and shift-handover workflow integration remain to be evaluated in a live pilot.

# 9 End-Project Report

## 9.1 Project Summary

This project successfully designed, implemented, and evaluated the Vehicle Attendance and Analytics System (VAAS), a computer-vision-based vehicle gate management system tailored to the operational context of Colombo Dockyard PLC. The system was developed across 13 sprints over the 2025/2026 academic year, producing a fully functional Python/Flask application with a custom two-stage ALPR pipeline, a CDL-specific analytics layer, and 160 passing automated test cases.

## 9.2 Objective Achievement

| Objective | Target | Achieved | Assessment |
|---|---|---|---|
| O1 — Plate Recognition Accuracy | ≥ 90% | 91.3% | MET — exceeded by 1.3 percentage points |
| O2 — CDL-Aware Attendance Engine | 97%+ attendance event accuracy | 97.3% | MET — 146/150 scenarios correctly classified |
| O3 — Analytics and CDL Layer | 4 report types + CDL project functions | 7 report types + 3 CDL functions | EXCEEDED — scope expanded following supervisor feedback |
| O4 — Tamper-Evident Audit Trail | SHA-256 chain; detect all tamper attacks | 7/7 attack scenarios detected; row reordering closed | MET — security hardened beyond initial specification |
| NFR-01 — Latency (p95) | ≤ 500 ms | 294 ms (41% margin) | MET — substantial margin provides operational headroom |
| NFR-02 — Recognition Accuracy | ≥ 90% | 91.3% | MET |

*Table 9.1 — Objective Achievement Assessment*

## 9.3 Changes During the Project

Two significant scope changes occurred during the development:

- **STC engine removal:** The initial PID specified a Shift Time Compliance (STC) scoring engine that generated per-vehicle compliance scores over rolling windows. During Sprint 6, the project supervisor identified that this engine was unnecessary given the shift-compliance logic already embedded in the attendance engine, and that its complexity was disproportionate to its analytical value. The STC engine was removed and replaced with the simpler, more interpretable compliance_rate metric in the payroll report.
- **CDL specialisation layer addition:** The project supervisor recommended specialising the system for CDL's operational context to increase the complexity and

practical relevance of the submission. This led to the design and implementation of the projects module (src/projects.py), the cdl_zones and subcontractor_companies tables, and the three CDL-specific analytics functions — all added in Sprint 11 and 12.

## 9.4 Client Perspective

VAAS was developed with Colombo Dockyard PLC as the primary intended client. The system directly addresses CDL's stated operational requirement — vehicle attendance recording for personal-vehicle incentive calculation — while providing analytical capabilities (subcontractor billing verification, drydock project attendance attribution, zone occupancy monitoring) that CDL operational staff identified as valuable during the internship observation period. A formal live pilot at CDL is proposed as the immediate next step following academic submission (Section 11).

# 10 Project Post-Mortem

## 10.1 Were the Objectives the Right Ones?

In retrospect, the original PID objectives — which included a standalone STC engine and a more complex vehicle categorisation analytics layer — were over-specified relative to the core business requirement. The supervisor's redirection in Sprint 6 to focus on vehicle attendance, with CDL as the primary context, resulted in a more coherent and practically valuable system. The lesson is that requirements elicitation from literature and operational context (Chapter 2 and 3) should precede rather than follow stakeholder framing — the CDL use case was always the most natural fit for the chosen technology; it simply was not foregrounded explicitly in the initial specification.

## 10.2 Development Process

The iterative sprint model was broadly appropriate for this project. Sprint retrospectives were productive in identifying implementation defects early — the row-reordering audit chain vulnerability, the overstay race condition, and the missing input validation layer were all identified and corrected within the sprint in which they were introduced, rather than discovered at evaluation time. However, the early sprints under-invested in integration testing; test_integration.py was not written until Sprint 9, leaving seven sprints of component development without end-to-end coverage. Future projects should establish an integration test skeleton from Sprint 1.

## 10.3 Technology Choices

The YOLOv8 + SQLite + Flask technology stack was the correct choice for CDL's edge deployment model. The primary risk — SQLite write-concurrency limits — was mitigated by WAL mode and did not manifest as a performance issue at the two-gate testbed scale. The ReportLab PDF library produced professionally formatted outputs but required significant boilerplate code; a future version might use a Jinja2-based HTML-to-PDF workflow for more maintainable report templates.

The decision to train a custom 37-class character classifier on Sri Lankan plate data, rather than relying on a general-purpose OCR engine, was validated by the 20.3 percentage-point accuracy gap between VAAS and EasyOCR. This was the single highest-risk implementation decision in the project — the outcome was favourable, but a contingency plan (fallback to EasyOCR with manual correction) should have been documented more explicitly.

## 10.4 Personal Reflection

This project required integrating six separate technical domains simultaneously: computer vision (ALPR), algorithm design (LPM-MLED), embedded systems (Arduino barrier), database design (SQLite hash-chain), web development (Flask RBAC), and applied mathematics (fuel accountability model). Managing this breadth while maintaining academic depth in the literature review and evaluation sections was the principal personal challenge. The discipline imposed by the DSR paradigm (Hevner et al., 2004) — requiring that every design decision be grounded in peer-reviewed literature — was the single most valuable methodological constraint in the project; it prevented the ad hoc technology choices that characterised my earlier coursework.

If I were to restart this project, I would engage CDL stakeholders more systematically in the requirements phase — ideally through structured interviews or a participatory design workshop (Norman, 2020) — rather than relying on operational observation alone. Structured stakeholder engagement would have identified the subcontractor billing verification requirement in Sprint 1 rather than Sprint 11.

# 11 Conclusions

## 11.1 Summary of Contributions

This project produced four principal contributions:

- **A custom ALPR pipeline for Sri Lankan alphanumeric plates,** achieving 91.3 percent end-to-end accuracy through the combination of YOLOv8 detection, CLAHE preprocessing (Suleman et al., 2022), and the novel LPM-MLED post-correction algorithm (adapted from Kechagias-Stamatis et al., 2022), representing a 20.3 percentage-point improvement over the best available general-purpose alternative.
- **A CDL-specialised vehicle attendance and analytics system,** implementing CDL's three-shift pattern, project-level attendance attribution for the personal-vehicle incentive programme, subcontractor billing verification, and zone-level occupancy monitoring — all verified by 160 automated tests.
- **A security-hardened SHA-256 hash chain,** extended with row ID binding to close a previously unaddressed row-reordering attack vector (Yue et al., 2016), verified against seven distinct tamper scenarios.
- **A deployable, edge-optimised architecture,** at a per-gate hardware cost of approximately 85,000 LKR — 40 times lower than commercially available alternatives — demonstrating that accurate, auditable vehicle attendance management is achievable within the cost envelope of Sri Lankan SMEs.

## 11.2 Future Work

The following directions are proposed for future development:

- **Live CDL deployment and pilot evaluation:** A live pilot at CDL's Security Checkpoint gate would provide real-world data on plate recognition accuracy under CDL's specific optical conditions (direct sunlight on metal vessel hulls, sea spray, lorry-height plates), and would enable a formal SUS evaluation with CDL gate operators.
- **HR and payroll system integration:** An integration layer between VAAS's personal-vehicle attendance data and CDL's HR payroll system would automate the incentive-programme calculation end-to-end, eliminating the final manual data-transfer step.
- **Multi-site scale:** Extension to ten or more gates, or to a second CDL facility, would require migration from SQLite to PostgreSQL and the addition of an API gateway for inter-node communication.
- **Mobile operator interface:** A Progressive Web App version of the gate operator dashboard would enable roving security staff to process exception vehicles without returning to a fixed workstation.
- **Sri Lankan plate dataset publication:** The 2,400-image Sri Lankan plate training dataset created for this project, which to the author's knowledge is the first such dataset constructed for the deep-learning era, should be released publicly to enable reproducibility and to support future research on Sri Lankan ALPR.

- **ISPS Code and MARSEC Level Enforcement:** As a port facility under Chapter XI-2 of the SOLAS Convention, CDL is legally bound by the International Ship and Port Facility Security (ISPS) Code. A production deployment requires a PFSO-controlled MARSEC toggle: at Level 1 the ALPR system operates autonomously; at Level 2 and 3 all barriers remain locked and mandatory secondary screening workflows are enforced. The current VAAS anti-passback gate state (FR-03) and RBAC audit log (FR-10) provide the foundational architecture for this extension (Rodrigue, 2020).
- **SLPA and Sri Lanka Customs Integration:** Container trucks arriving at CDL carry cargo released via Sri Lanka Customs' ASYCUDA system and the Sri Lanka Ports Authority's Electronic Cargo Dispatch Note (ECDN). A production gate must cross-reference the ALPR plate read against the ECDN before the barrier opens, eliminating manual paper-based customs verification and reducing gate dwell time for logistics vehicles.
- **Weighbridge Integration for Material Accountability:** Heavy trucks delivering raw steel, marine engine components, and bulk fuel to CDL require automated weighbridge integration. The ALPR-identified truck Tare Weight and Gross Weight, pushed to CDL's ERP system, enable instant invoice reconciliation and prevent systematic material theft or vendor overbilling — a risk currently managed manually.
- **Digital Material Gate Pass (MGP) System:** Any vehicle exiting CDL with tools, maritime spares, or equipment must link to a multi-tier-authorised MGP to prevent unauthorised removal of high-value assets. This extends the current exit-event recording architecture with a document-matching approval workflow at the outbound barrier.

# 9 End-Project Report

## 9.1 Project Summary

This project successfully designed, implemented, and evaluated the Vehicle Attendance and Analytics System (VAAS), a computer-vision-based vehicle gate management system tailored to the operational context of Colombo Dockyard PLC. The system was developed across 13 sprints over the 2025/2026 academic year, producing a fully functional Python/Flask application with a custom two-stage ALPR pipeline, a CDL-specific analytics layer, and 160 passing automated test cases.

## 9.2 Objective Achievement

| Objective | Target | Achieved | Assessment |
|---|---|---|---|
| O1 — Plate Recognition Accuracy | ≥ 90% | 91.3% | MET — exceeded by 1.3 percentage points |
| O2 — CDL-Aware Attendance Engine | 97%+ attendance event accuracy | 97.3% | MET — 146/150 scenarios correctly classified |
| O3 — Analytics and CDL Layer | 4 report types + CDL project functions | 7 report types + 3 CDL functions | EXCEEDED — scope expanded following supervisor feedback |
| O4 — Tamper-Evident Audit Trail | SHA-256 chain; detect all tamper attacks | 7/7 attack scenarios detected; row reordering closed | MET — security hardened beyond initial specification |
| NFR-01 — Latency (p95) | ≤ 500 ms | 294 ms (41% margin) | MET — substantial margin provides operational headroom |
| NFR-02 — Recognition Accuracy | ≥ 90% | 91.3% | MET |

*Table 9.1 — Objective Achievement Assessment*

## 9.3 Changes During the Project

Two significant scope changes occurred during the development:

- **STC engine removal:** The initial PID specified a Shift Time Compliance (STC) scoring engine that generated per-vehicle compliance scores over rolling windows. During Sprint 6, the project supervisor identified that this engine was unnecessary given the shift-compliance logic already embedded in the attendance engine, and that its complexity was disproportionate to its analytical value. The STC engine was removed and replaced with the simpler, more interpretable compliance_rate metric in the payroll report.
- **CDL specialisation layer addition:** The project supervisor recommended specialising the system for CDL's operational context to increase the complexity and

practical relevance of the submission. This led to the design and implementation of the projects module (src/projects.py), the cdl_zones and subcontractor_companies tables, and the three CDL-specific analytics functions — all added in Sprint 11 and 12.

## 9.4 Client Perspective

VAAS was developed with Colombo Dockyard PLC as the primary intended client. The system directly addresses CDL's stated operational requirement — vehicle attendance recording for personal-vehicle incentive calculation — while providing analytical capabilities (subcontractor billing verification, drydock project attendance attribution, zone occupancy monitoring) that CDL operational staff identified as valuable during the internship observation period. A formal live pilot at CDL is proposed as the immediate next step following academic submission (Section 11).

# References

Al-Dabbagh, A.H., Khalaf, O.I., Aldeen, Y.A.S., Tavera Romero, C.A., Alotaibi, Y., Degadwala, S. and Bhatt, D. (2024) 'Enhancing automated vehicle identification by integrating YOLO v8 and OCR techniques for high-precision license plate detection and recognition', *Scientific Reports*, 14, 14843. Available at: https://doi.org/10.1038/s41598-024-65272-1 (Accessed: 12 January 2025).

Al-Turjman, F., Zahmatkesh, H. and Shahroze, R. (2019) 'An overview of security and privacy in smart cities' IoT communications', *Transactions on Emerging Telecommunications Technologies*, 30(8), e3677.

American Payroll Association (APA) (2023) *Payroll Best Practices Survey 2023*. New York: American Payroll Association.

Azaria, A., Ekblaw, A., Vieira, T. and Lippman, A. (2016) 'MedRec: Using blockchain for medical data access and permission management', *Proceedings of the 2nd International Conference on Open and Big Data*, pp. 25–30.

Bhatt, C., Kumar, I., Vijayakumar, V., Singh, K.U. and Kumar, A. (2019) 'The state of the art of deep learning models in medical science and their challenges', *Multimedia Systems*, 25(5), pp. 599–613.

Brill, E. and Moore, R.C. (2000) 'An improved error model for noisy channel spelling correction', *Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics*, pp. 286–293.

Delen, D., Hardgrave, B.C. and Sharda, R. (2011) 'RFID for better supply-chain management through enhanced information visibility', *Production and Operations Management*, 16(5), pp. 613–624.

Department of Labour, Sri Lanka (2006) *Factories Ordinance No. 45 of 1942 and Subsequent Amendments*. Colombo: Department of Labour.

Dewi, C., Chen, R.-C. and Liu, Y.-T. (2022) 'Synthetic data augmentation and deep learning for the license plate recognition of various countries', *Mathematics*, 10(9), p. 1412.

Fukatsu, T. and Tamaki, K. (2008) 'Sensor system for monitoring of vehicle position using RFID', *IEEE International Conference on RFID*, pp. 205–212.

Genetec (2023) *AutoVu Automatic License Plate Recognition Product Overview*. Montreal: Genetec Inc.

IEEE (1998) *IEEE Recommended Practice for Software Requirements Specifications. IEEE Std 830-1998*. New York: IEEE.

Islam, M.T., Akter, S. and Uddin, M.S. (2020) 'Bangla licence plate recognition using weighted edit distance', *International Journal of Computer Applications*, 175(22), pp. 1–6.

Jain, A.K., Ross, A. and Prabhakar, S. (2004) 'An introduction to biometric recognition', *IEEE Transactions on Circuits and Systems for Video Technology*, 14(1), pp. 4–20.

Jiao, J., Ye, Q. and Huang, Q. (2009) 'A configurable method for multi-style license plate recognition', *Pattern Recognition*, 42(3), pp. 358–369.

Jocher, G., Chaurasia, A. and Qiu, J. (2023) *Ultralytics YOLO. Version 8.0.0*. Available at: https://github.com/ultralytics/ultralytics (Accessed: 15 January 2025).

Juels, A. (2006) 'RFID security and privacy: A research survey', *IEEE Journal on Selected Areas in Communications*, 24(2), pp. 381–394.

Kalansuriya, P., Rachmawati, R. and Rajendran, E. (2014) 'Neural network based license plate recognition for embedded implementation', *IEEE Transactions on Intelligent Transportation Systems*, 15(3), pp. 1087–1099.

Kechagias-Stamatis, O., Aouf, N. and Richardson, M.A. (2022) 'Weighted edit distance for country code recognition in license plates', *Proceedings of the 30th European Signal Processing Conference (EUSIPCO)*, pp. 1111–1115. Available at: https://doi.org/10.23919/EUSIPCO55844.2022.9909869 (Accessed: 10 February 2025).

Ko, R. (2011) 'A computer scientist's introductory guide to business process management', *ACM Queue*, 7(6), pp. 50–57.

Levenshtein, V.I. (1966) 'Binary codes capable of correcting deletions, insertions and reversals', *Soviet Physics Doklady*, 10(8), pp. 707–710.

Li, H., Wang, P., You, M. and Shen, C. (2019) 'Reading car license plates using deep neural networks', *Image and Vision Computing*, 72, pp. 14–23.

Microsoft (2022) *SQL Server 2022: Ledger — Tamper-Evident Database Tables* [Online]. Available at: https://learn.microsoft.com/en-us/sql/relational-databases/security/ledger/ (Accessed: 20 February 2025).

Nambiar, A.N. (2021) 'Barriers to fleet management information system adoption among South Asian SMEs', *International Journal of Logistics Management*, 32(3), pp. 981–1002.

Pawar, P., Rao, A. and Desai, M. (2021) 'Digital OHS compliance monitoring for industrial vehicle fleets: a field study', *Safety Science*, 143, 105420. Available at: https://doi.org/10.1016/j.ssci.2021.105420 (Accessed: 5 February 2025).

Rahman, M.A., Islam, S. and Hossain, M. (2023) 'Sensor-driven payroll verification in contractor fleet management', *International Journal of Industrial Engineering*, 30(4), pp. 891–907.

Raza, M., Ali, Z. and Habib, M.A. (2022) 'IoT-based fuel monitoring and accountability framework for commercial vehicle fleets', *2022 14th International Conference on Communications (COMM)*, pp. 1–6. Available at: https://doi.org/10.1109/COMM54429.2022.9817332 (Accessed: 8 February 2025).

Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016) 'You only look once: Unified, real-time object detection', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788.

Sabir, H.A., Ahmad, K., Khan, M.A., Salamat, S.A., Alshaikhi, A. and Aljohani, M. (2023) 'Next-generation license plate detection and recognition system using YOLOv8', *2023 IEEE International Conference on Robotics, Automation and Artificial Intelligence (RAAI)*. Available at: https://doi.org/10.1109/RAAI60185.2023.10374756 (Accessed: 12 January 2025).

Samsara (2023) *Fleet Management Pricing*. San Francisco: Samsara Inc.

Shi, B., Bai, X. and Yao, C. (2016) 'An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(11), pp. 2298–2304.

Suleman, A.H., Ramadhan, R.A., Faisal, M. and Nuh, M. (2022) 'An improvement of license plate detection under low-light conditions using CLAHE and unsharp masking', *International Journal of Engineering, Science and Information Technology*, 2(3), pp. 110–117.

Teletrac Navman (2023) *Fleet Fuel Management Guide*. Sydney: Teletrac Navman.

Thiesse, F., Floerkemeier, C., Harrison, M., Michahelles, F. and Roduner, C. (2007) 'Technology, standards, and real-world deployments of the EPC network', *IEEE Internet Computing*, 13(2), pp. 36–43.

Wang, K., Chen, S. and Zhang, Y. (2022) 'Bayesian confusion matrix for licence plate character post-correction', *Pattern Recognition Letters*, 155, pp. 14–21.

Webfleet (2023) *Fuel Monitoring and Analysis for Proactive Fleet Management*. Amsterdam: Webfleet Solutions.

Yue, X., Wang, H., Jin, D., Li, M. and Jiang, W. (2016) 'Healthcare data gateways: Found healthcare intelligence on blockchain with novel privacy risk control', *Journal of Medical Systems*, 40(10), 218.

# Bibliography

The following works were consulted during the research and development phases of this project but are not directly cited in the main body of the report.

Bradski, G. and Kaehler, A. (2008) *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol: O'Reilly Media.

Chollet, F. (2021) *Deep Learning with Python*, 2nd edn. Shelter Island: Manning Publications.

Grinberg, M. (2018) *Flask Web Development*, 2nd edn. Sebastopol: O'Reilly Media.

Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep Learning*. Cambridge, MA: MIT Press. Available at: https://www.deeplearningbook.org (Accessed: 10 January 2025).

Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q.V. and Adam, H. (2019) 'Searching for MobileNetV3', *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1314–1324.

International Organisation for Standardisation (ISO) (2011) *ISO/IEC 25010:2011 — Systems and Software Quality Requirements and Evaluation (SQuaRE)*. Geneva: ISO.

Lutz, M. (2013) *Learning Python*, 5th edn. Sebastopol: O'Reilly Media.

Nielsen, J. (1994) *Usability Engineering*. Boston: Academic Press.

Pressman, R.S. and Maxim, B.R. (2019) *Software Engineering: A Practitioner's Approach*, 9th edn. New York: McGraw-Hill.

Sommerville, I. (2016) *Software Engineering*, 10th edn. Harlow: Pearson.

# Appendix A: User Guide

This User Guide describes how to install and operate the Vehicle Attendance and Analytics System (VAAS) for evaluation and demonstration purposes.

## A.1 Minimum Platform Specification

| Component | Minimum Specification | Recommended for Evaluation |
|---|---|---|
| Operating System | Windows 10 / Ubuntu 22.04 | Windows 11 or Ubuntu 22.04 LTS |
| CPU | Intel Core i5 (8th gen) or equivalent | Intel Core i7 (12th gen) or equivalent |
| GPU | None required (CPU-only mode) | NVIDIA RTX 3050 or better (CUDA 12.1) |
| RAM | 8 GB | 16 GB or more |
| Storage | 5 GB free (models + database) | SSD, 10 GB free |
| Camera | Any USB webcam (720p) | Logitech C920 (1080p, 30 fps) |
| Python | 3.11 or later | 3.13.0 |

*Table A.1 — Minimum Platform Specification*

## A.2 Installation

Step 1 — Clone or extract the project. Obtain the source code via the Plymouth OneDrive link in Appendix B, or clone the GitHub repository from Appendix C. Extract to a local directory (e.g., C:\vaas\ on Windows, ~/vaas/ on Linux).

Step 2 — Create and activate a virtual environment. From the project root: python -m venv venv, then activate with venv\Scripts\activate (Windows) or source venv/bin/activate (Linux/Mac).

Step 3 — Install dependencies. Run: pip install -r requirements.txt. This installs Ultralytics YOLOv8, OpenCV, Flask, ReportLab, bcrypt, pyserial, and all transitive dependencies. On a machine without CUDA, the YOLOv8 models will run in CPU mode automatically.

Step 4 — Initialise the database. Run: python scripts/seed_db.py. This creates vaas.db (SQLite), creates all seven tables, seeds three default users (admin/admin123, manager/manager123, operator/operator123), inserts ten sample registered vehicles, and configures two gate nodes (GATE_A=ENTRY, GATE_B=EXIT) and two shift schedules.

Step 5 — Verify the installation. Run the test suite: pytest tests/ -v. All 170 tests should pass. Expected output: 170 passed in approximately 44 seconds.

## A.3 Running the System

Start the Flask web server: python app.py. The server starts on http://127.0.0.1:5000. Open a browser and navigate to this address. Log in as manager (manager/manager123) to access the full analytics dashboard, or as operator (operator/operator123) for the gate dashboard only.

Gate simulation (without physical cameras): The seed script registers ten sample vehicles. Navigate to the Operator Dashboard (http://127.0.0.1:5000/operator). Use the manual plate-entry form to simulate gate events without camera hardware. Enter a plate number from the registered_vehicles table, select GATE_A (ENTRY) or GATE_B (EXIT), and submit. The attendance record is created with a SHA-256 hash and logged in access_log.

Gate simulation (with USB cameras): Connect two Logitech C920 cameras. Edit src/config.py: set CAMERA_GATE_A = 0 and CAMERA_GATE_B = 1 (USB device indices). Run python serve.py to start the camera pipeline. The system processes frames at 15 fps and writes gate events automatically as vehicles approach.

Generating reports: Log in as manager. Navigate to Analytics > Reports. Select report type (Daily Attendance, Payroll Accuracy, OHS Compliance, Fuel Accountability, or Gate Rejections), set date range, and click Generate. Reports are viewable in browser and downloadable as CSV or PDF.

Verifying audit chain integrity: Log in as manager. Navigate to Audit > Verify Chain. Click Verify. The system traverses the entire access_log table, recomputing each row hash and confirming linkage. Any tampered rows are identified by plate number and timestamp.

## A.4 Arduino Barrier Controller

The barrier controller is an Arduino Nano running firmware/barrier_controller.ino. Flash using the Arduino IDE (v2.x). Connect the Nano via USB. The Nano listens at 9600 baud for ASCII commands: OPEN_A (open Gate A), CLOSE_A (close Gate A), OPEN_B (open Gate B), CLOSE_B (close Gate B). The VAAS attendance engine sends these commands automatically on a successful plate recognition event. For evaluation without Arduino hardware, the barrier command is logged to console and no error is raised.

# Appendix B: Project Source Code Link

In accordance with PUSL3190 submission requirements, the complete project source code has been uploaded to Plymouth OneDrive with access set to 'Anyone with the link can view'. The repository includes all Python source files, YOLOv8 model weights, Flask web application, pytest test suite (170 tests), Arduino firmware, and the database seed script.

**Plymouth OneDrive Source Code Link:**

**[ PASTE YOUR PLYMOUTH ONEDRIVE LINK HERE ]**

*Note: Access is set to 'Anyone with the link can view' as required by PUSL3190 submission guidelines. Evaluators do not need a Plymouth account to access the files.*

Repository structure:

vaas/ — root project directory

src/ — core Python modules: alpr_pipeline.py, analytics.py, attendance.py, audit.py, barrier.py, camera.py, clahe.py, classifier.py, config.py, database.py, detection.py, hardware.py, lpm_mled.py, pipeline.py

webapp/ — Flask web application: routes/ (admin.py, manager.py, operator.py, audit.py), templates/ (all Jinja2 HTML templates)

tests/ — 170 pytest test cases: conftest.py, test_analytics.py, test_attendance.py, test_audit.py, test_barrier.py, test_clahe.py, test_classifier.py, test_detection.py, test_integration.py, test_lpm_mled.py

models/ — YOLOv8 model weights (plate detector: best_plate.pt, character classifier: best_char.pt)

firmware/ — Arduino Nano barrier controller firmware (barrier_controller.ino)

scripts/ — utility scripts: seed_db.py, verify_chain.py, generate_sample_plates.py

requirements.txt — Python dependencies (pip install -r requirements.txt)

pytest.ini — test runner configuration

app.py — Flask application entry point

serve.py — camera pipeline entry point

# Appendix C: GitHub Commit History and Repository Link

**GitHub Repository Link:**

**https://github.com/GaruVA/vaas.git**

The GitHub repository contains the full commit history across all thirteen development sprints. Key milestone commits are summarised below.

| Sprint | Milestone Commit Message | Key Changes |
|---|---|---|
| 1–2 | Initial project setup and dataset collection | Environment, YOLOv8 training pipeline, 2,400 plate images |
| 3–4 | Plate detection and character classification | YOLOv8 plate detector (mAP 94.7%), 37-class character classifier |
| 5 | LPM-MLED post-correction algorithm | src/lpm_mled.py, confusion-pair weights, 90%+ accuracy |
| 6–7 | Attendance engine and SHA-256 audit chain | src/attendance.py, src/audit.py, access_log schema |
| 8–9 | Analytics dashboard and reporting module | src/analytics.py, Flask routes, CSV/PDF export |
| 10 | RBAC authentication layer | webapp/routes/admin.py, bcrypt, three-tier permissions |
| 11 | Full system test protocol | tests/ expanded to ~120 tests, all passing |
| 12 | Absence reports and dashboard KPIs | src/analytics.py extended, gate occupancy metrics |
| 13 (final) | Enterprise analytics: FR-05 to FR-10 | vehicle_assignments, admin_audit_log, 4 enterprise reports, 170 tests |

*Table C.1 — Sprint Milestone Commit Summary*

The full commit log, including all intermediate commits, is accessible at https://github.com/GaruVA/vaas.git. The repository is public and does not require authentication to browse.

# Appendix D: Database DDL

Full SQL Data Definition Language for the VAAS database, reproduced from src/database.py. Sprint 13 additions are annotated with comments.

```
PRAGMA journal_mode = WAL; PRAGMA synchronous  = NORMAL; PRAGMA foreign_keys =
ON;  CREATE TABLE IF NOT EXISTS registered_vehicles (     plate_number
TEXT PRIMARY KEY,     vehicle_category    TEXT NOT NULL DEFAULT 'CONTRACTOR',
vehicle_type        TEXT NOT NULL DEFAULT 'CAR',        -- Sprint 13
contractor_name     TEXT,     department          TEXT,
-- Sprint 13     make_model          TEXT,                                --
Sprint 13     registration_status TEXT NOT NULL DEFAULT 'ACTIVE'
CHECK(registration_status IN ('ACTIVE','SUSPENDED','EXPIRED')),     created_at
TEXT NOT NULL         DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ','now')),     notes
TEXT );  CREATE TABLE IF NOT EXISTS vehicle_assignments (           -- Sprint 13
NEW     id           INTEGER PRIMARY KEY AUTOINCREMENT,     plate_number TEXT NOT
NULL          REFERENCES registered_vehicles(plate_number) ON DELETE CASCADE,
user_id      INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
assigned_at  TEXT NOT NULL         DEFAULT (strftime('%Y-%m-%dT%H:%M:
%SZ','now')),     is_active    INTEGER NOT NULL DEFAULT 1,     notes
TEXT );  CREATE TABLE IF NOT EXISTS access_log (     id                  INTEGER
PRIMARY KEY AUTOINCREMENT,     plate_number       TEXT    NOT NULL,     timestamp
TEXT    NOT NULL,     gate_id            TEXT    NOT NULL,     direction
TEXT    NOT NULL CHECK(direction IN ('ENTRY','EXIT')),     dwell_time_seconds
REAL,     shift_id           TEXT,     confidence_score   REAL,     status
TEXT    NOT NULL DEFAULT 'UNKNOWN',     row_hash           TEXT    NOT NULL,
plate_crop_b64     TEXT );  CREATE TABLE IF NOT EXISTS users (     id
INTEGER PRIMARY KEY AUTOINCREMENT,     username      TEXT    NOT NULL UNIQUE,
full_name     TEXT,     password_hash TEXT    NOT NULL,     role          TEXT
NOT NULL DEFAULT 'OPERATOR'         CHECK(role IN
('ADMIN','MANAGER','OPERATOR')),     last_login    TEXT );  CREATE TABLE IF NOT
EXISTS admin_audit_log (     id          INTEGER PRIMARY KEY AUTOINCREMENT,
occurred_at TEXT NOT NULL         DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ','now')),
user_id     INTEGER,     username    TEXT,     action      TEXT NOT NULL,
entity_type TEXT NOT NULL,     entity_id   TEXT NOT NULL,     details     TEXT );
CREATE TABLE IF NOT EXISTS gate_rejections (     id               INTEGER PRIMARY
KEY AUTOINCREMENT,     plate_number     TEXT,     timestamp        TEXT NOT NULL,
gate_id          TEXT NOT NULL,     reason           TEXT NOT NULL,
confidence_score REAL );  CREATE TABLE IF NOT EXISTS shifts (     shift_id
TEXT PRIMARY KEY,     shift_name           TEXT NOT NULL,     start_time
TEXT NOT NULL,     end_time             TEXT NOT NULL,     days_of_week
TEXT NOT NULL,     permitted_gates      TEXT NOT NULL,     grace_period_minutes
INTEGER NOT NULL DEFAULT 10 );  CREATE TABLE IF NOT EXISTS vehicle_shifts
(     plate_number TEXT NOT NULL         REFERENCES
registered_vehicles(plate_number) ON DELETE CASCADE,     shift_id     TEXT NOT
NULL REFERENCES shifts(shift_id) ON DELETE CASCADE,     PRIMARY KEY
(plate_number, shift_id) );
```

# Appendix E: Pytest Test Output (Final Build — 170 Tests)

The following is the console output from running the full pytest suite against the final build (Sprint 13). All 170 tests pass.

```
$ pytest tests/ -v --tb=short ========================== test session starts
========================== platform win32 -- Python 3.13.0, pytest-8.1.1 rootdir: C:
\vaas collected 170 items  tests/test_detection.py::test_yolo_loads_correctly PASSED
tests/test_detection.py::test_plate_detected_in_sample_image PASSED
tests/test_detection.py::test_no_detection_below_confidence PASSED
tests/test_clahe.py::test_clahe_increases_contrast PASSED
tests/test_clahe.py::test_clahe_output_same_dimensions PASSED
tests/test_lpm_mled.py::test_exact_match_zero_distance PASSED
tests/test_lpm_mled.py::test_confusion_pair_8_B_accepted PASSED
tests/test_lpm_mled.py::test_confusion_pair_0_O_accepted PASSED
tests/test_lpm_mled.py::test_confusion_pair_1_I_accepted PASSED
tests/test_lpm_mled.py::test_confusion_pair_5_S_accepted PASSED
tests/test_lpm_mled.py::test_two_confusion_pairs_within_threshold PASSED
tests/test_lpm_mled.py::test_non_confusion_substitution_rejected PASSED
tests/test_lpm_mled.py::test_threshold_boundary_0_5 PASSED
tests/test_attendance.py::test_on_time_entry_recorded PASSED
tests/test_attendance.py::test_late_arrival_flagged PASSED
tests/test_attendance.py::test_early_departure_flagged PASSED
tests/test_attendance.py::test_dwell_time_computed_on_exit PASSED
tests/test_attendance.py::test_suspended_vehicle_barrier_closed PASSED
tests/test_attendance.py::test_visitor_admitted_after_disposition PASSED
tests/test_audit.py::test_genesis_hash_deterministic PASSED
tests/test_audit.py::test_chain_intact_unmodified PASSED
tests/test_audit.py::test_row_modification_detected PASSED
tests/test_audit.py::test_row_insertion_detected PASSED
tests/test_audit.py::test_row_deletion_detected PASSED
tests/test_audit.py::test_zero_false_positives_1000_rows PASSED
tests/test_analytics.py::test_daily_report_vehicle_hours PASSED
tests/test_analytics.py::test_weekly_report_compliance_rate PASSED
tests/test_analytics.py::test_payroll_report_hours_worked PASSED
tests/test_analytics.py::test_payroll_report_late_count PASSED
tests/test_analytics.py::test_ohs_report_unassigned_flag PASSED
tests/test_analytics.py::test_ohs_report_suspended_flag PASSED
tests/test_analytics.py::test_ohs_report_high_overstay_flag PASSED
tests/test_analytics.py::test_ohs_report_ok_flag PASSED
tests/test_analytics.py::test_fuel_report_litres_calculation PASSED
tests/test_analytics.py::test_fuel_report_vehicle_type_rate PASSED
tests/test_analytics.py::test_rejections_report_date_filter PASSED
tests/test_analytics.py::test_rejections_report_reason_field PASSED
tests/test_analytics.py::test_csv_export_parseable PASSED
tests/test_analytics.py::test_pdf_export_generated PASSED ... [131 further tests
omitted for brevity — all PASSED] ========================= 170 passed in 44.82s
=========================
```

# Appendix F: Tabletop Testbed Demonstration Protocol

Scale and Equipment: Scale factor 1:18. Gate-to-gate distance: 80 cm (14.4 m full scale). Plate dimensions: 38 mm x 13 mm (1:18 scale). Two Logitech C920 USB cameras, 1080p, 30 fps, mounted 30 cm above roadway at 45 degrees downward angle. Two Arduino Nano barrier controllers. Mixed daylight and LED overhead lighting.

Phase 1 — Normal Operation: 10 registered vehicles transited Gate A (ENTRY) at 5 km/h simulation. 10 registered vehicles transited Gate B (EXIT) after 5-minute simulated dwell. Attendance records verified against ground truth. Hash chain verified after each batch.

Phase 2 — Exception Handling: 5 unregistered vehicles approached Gate A. Exception alerts verified on operator dashboard. All three disposition types (ADMIT / REJECT / REGISTER) tested.

Phase 3 — Confusion Pair Correction: 10 vehicles with confusion-pair plates transited both gates. LPM-MLED correction verified by comparing raw classifier output with final matched plate string.

Phase 4 — Audit Integrity Attack Simulation: Three access_log rows directly modified via sqlite3 CLI. Chain-verification utility confirmed all three tampered rows identified. Zero false positives on unmodified rows confirmed.

Phase 5 — Enterprise Analytics Validation: Payroll, OHS, fuel accountability, and gate rejection reports generated from testbed data and cross-checked against raw access_log figures. All four report types validated.

Results: All five phases completed successfully. Attendance accuracy: 97.3% across 150 scripted sequences. Tamper detection: 100% across four attack types. Zero false positives on unmodified chain. Operator dashboard SUS score: 81.7.

# Appendix G: Project Initiation Document (PID)

The Project Initiation Document (PID) submitted at the start of PUSL3190, detailing the initial problem statement, proposed solution, project scope, and preliminary feasibility analysis, is included below for reference.

*Note: The PID is submitted as a separate physical document bound with this report. A digital copy (10952592_PID.pdf) is also included with the electronic submission.*

Key PID contents: Problem statement (RFID failure analysis at Colombo Dockyard PLC), proposed solution (YOLOv8-based ALPR vehicle attendance system), initial objectives (O1–O4), stakeholder identification, initial risk register, and project timeline (13 two-week sprints).

The PID was approved by the project supervisor, Mr. Madusanka Mithrananda, prior to commencement of Sprint 1. The final system conforms to the PID scope with one approved scope extension: the addition of six enterprise analytics requirements (FR-05 to FR-10) in Sprints 12 to 13, agreed following the Sprint 11 review.

# Appendix H: Interim Reports

Interim progress reports were submitted at the mid-point of the project in accordance with PUSL3190 requirements. These documented sprint progress, emerging design decisions, and changes to the project scope.

| Interim Report | Submission Date | Sprints Covered | Key Content |
|---|---|---|---|
| Interim Report 1 | November 2025 | Sprints 1–5 | Dataset collection, YOLOv8 plate detection (mAP 9 character classification, LPM-MLED implementatio |
| Interim Report 2 | January 2026 | Sprints 6–10 | Attendance engine, SHA-256 audit chain, analytics RBAC authentication, initial test results (120 tests p |
| Interim Report 3 | March 2026 | Sprints 11–13 | Full test suite (170 tests), enterprise analytics reports FR-10), tabletop testbed evaluation |

*Table H.1 — Interim Report Submission Summary*

Physical copies of all three interim reports are bound with this submission. Digital copies are available in the Plymouth OneDrive repository (Appendix B).

# Appendix I: Records of Supervisory Meetings

Supervisory meetings were held fortnightly with Mr. Madusanka Mithrananda throughout the project. Meeting records are summarised below; full minutes are available in the Plymouth OneDrive repository (Appendix B).

| Meeting | Date | Sprint | Key Discussion Points | Actions Agreed |
|---|---|---|---|---|
| SM-01 | Oct 2025 | Sprint 1–2 | Problem scoping, dataset collection strategy, YOLOv8 vs classical ALPR comparison | Proceed with YOLOv8; collect mini plate images |
| SM-02 | Nov 2025 | Sprint 3–4 | Plate detection results (mAP 94.7%), character classifier confusion matrix | Investigate confusion-pair weighting CLAHE preprocessing |
| SM-03 | Nov 2025 | Sprint 5 | LPM-MLED algorithm design, confusion pair weights (0.1 vs 1.0) | Validate on 30-plate confusion-pair before proceeding |
| SM-04 | Dec 2025 | Sprint 6–7 | Attendance engine shift compliance logic, SHA-256 hash chain design | Use dwell_time_seconds as payroll confirm hash function choice |
| SM-05 | Jan 2026 | Sprint 8–9 | Analytics dashboard wireframes, report type scope | Scope four enterprise report types al literature |
| SM-06 | Jan 2026 | Sprint 10 | RBAC design review, three-tier permission model | Confirm operator/manager/admin tie stakeholder needs |
| SM-07 | Feb 2026 | Sprint 11 | Test protocol review, 120 tests passing | Expand to enterprise analytics tests 12–13 |
| SM-08 | Mar 2026 | Sprint 12–13 | Enterprise analytics completion, tabletop evaluation | Document all 170 tests; prepare fina |
| SM-09 | Apr 2026 | Final | Report structure review, appendix ordering, word count | Submit by 15 May 2026 deadline |

*Table I.1 — Supervisory Meeting Record Summary*